



第七章 图

杨震 计算机学院

7.1 图的基本概念

BUPT

图的基本概念

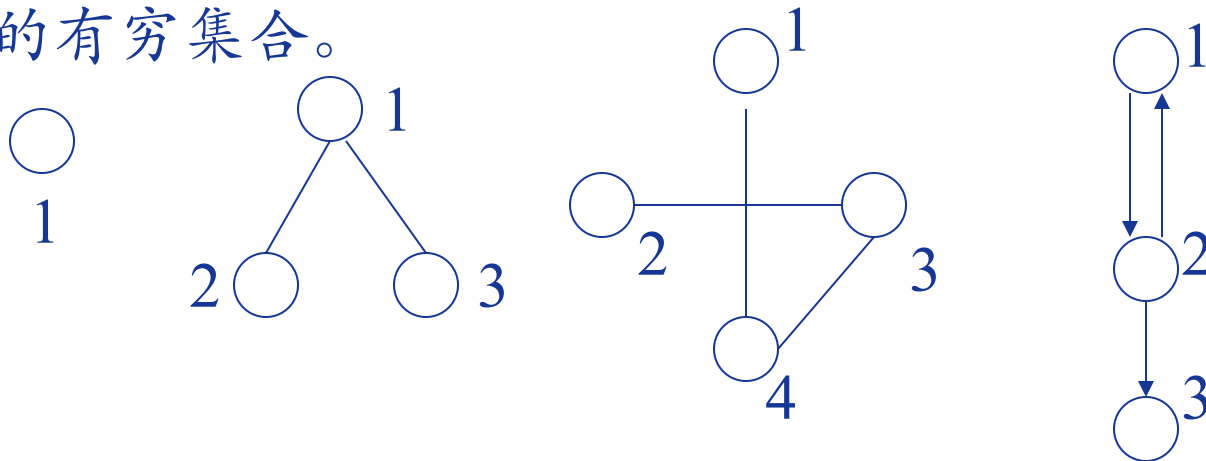
非线性结构，数据元素之间呈多对多的关系。

图的定义

$\text{Graph}=(V,E)$

V: 顶点(数据元素)的有穷非空集合。

E: 边的有穷集合。



BUPT SCST

图的相关术语

顶点 数据元素所构成的结点。

无向图 边的顶点偶对是无序的。

(v_i, v_j) 和 (v_j, v_i) 代表同一条**边**($i \neq j$)。

有向图 边的顶点偶对是有序的。

有向边 $\langle v_i, v_j \rangle$ 也称为**弧**。

v_i 是**弧尾**/初始点； v_j 是**弧头**/终端点。

(无向)完全图 每个顶点与其余顶点都有边的无向图。

顶点数为 n 时，边数 $e = n(n-1)/2$

有向完全图 每个顶点与其余顶点都有弧的有向图。

顶点数为 n 时，弧数 $e = n(n-1)$

思考：一无向图的顶点个数为 n ，则该图最多有 $(n(n-1)/2)$ 条边。

稀疏图 有很少边或弧的图。

稠密图 有较多边或弧的图。

权 图中的边或弧具有一定的大小的概念。

网 边/弧带权的图。

邻接 有边/弧相连的两个顶点之间的关系。

存在 (v_i, v_j) ，则称 v_i 和 v_j 互为**邻接点**；

存在 $\langle v_i, v_j \rangle$ ，则称 v_i **邻接到** v_j ， v_j **邻接于** v_i

关联(依附) 边/弧与顶点之间的关系。

存在 $(v_i, v_j)/\langle v_i, v_j \rangle$ ，则称该边/弧关联于 v_i 和 v_j

子图 对于图 $G=(V, E)$ 和 $G'=(V', E')$ ，如果 $V' \subseteq V$ ， $E' \subseteq E$ ，且 E' 关联的顶点都在 V' 中，则称 G' 是 G 的子图。

生成子图 由图的全部顶点和部分边组成的子图称为原图的生成子图。

杨龙驹的几把 需要被田欣睿撸撸

田欣睿 ylicat 的主人

顶点的度 与该顶点相关联的边的数目，记为 $D(v)$ 。

入度 $ID(v)$ ：有向图中，以该顶点为弧头的弧数目。

出度 $OD(v)$ ：有向图中，以该顶点为弧尾的弧数目。

顶点数 n 、边数 e 和度数之间的关系：
$$e = \frac{1}{2} \sum_{i=1}^n D(v_i)$$

求解方法归纳

利用无向图性质求解图中顶点个数时，经常使用如下特性：

①设图中度为 i 的顶点数为 n_i ，则对于无向**连通**图 $n_0=0$ 。

②若一个 n 顶点 e 条边无向**连通**图，所有顶点的度均小于 m ，则有： $n = n_0 + n_1 + n_2 + \dots + n_m$ ；所有顶点度之和 $= 2e$ ；所有顶点度之和 $= n_m * m + \dots + n_2 * 2 + n_1$ 。

实例：一个无向图共16条边，所有顶点度均小于5，度为4的顶点有3个，度为3的顶点有4个，其余顶点均小于3，则图中至少包含**11**个顶点。

解答：顶点数最少，则应为无向连通图， $n_0=0$ ， $n_4=3$ ， $n_3=4$ 。 $n = 7 + n_2 + n_1$ ；度之和 $= 4*3 + 3*4 + 2*n_2 + n_1 = 24 + 2n_2 + n_1 \leq 24 + 2*(n_2 + n_1) = 24 + 2*(n-7) = 10 + 2n$ 。又有度之和 $= 2e = 32$ ，因此 $n \geq 11$ 。

路径 接续的边构成的顶点序列。

路径长度 路径上边或弧的数目/权值之和。

回路(环) 第一个顶点和最后一个顶点相同的路径。

简单路径 除路径起点和终点可以相同外，其余顶点均不相同的路径。

简单回路(简单环) 除路径起点和终点相同外，其余顶点均不相同的路径。

思考： n 个顶点的无向完全图中,两个顶点之间简单路径数目为多少？

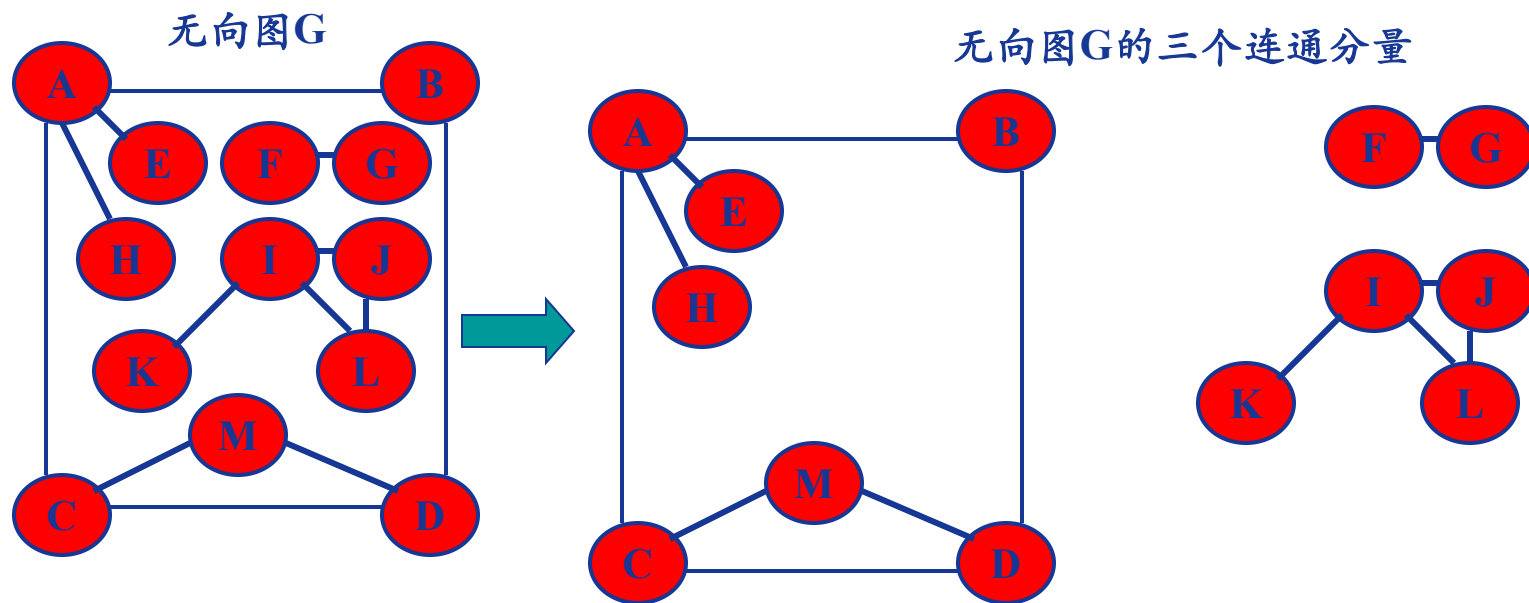
$$1 + A(n-2,1) + A(n-2,2) + \dots + A(n-2,n-2)$$

其中 $A(m,n)$ 是 m 取 n 的排列数。

连通图 无向图中，任何一对顶点间都存在路径。

连通分量 无向图中的极大连通子图，极大是指顶点数。

思考： n 个顶点组成的连通无向图，其边的个数至少为 $n-1$ 。



思考：一个有 n 个结点的无向图，最少有（1）个连通分量，
最多有（ n ）个连通分量。

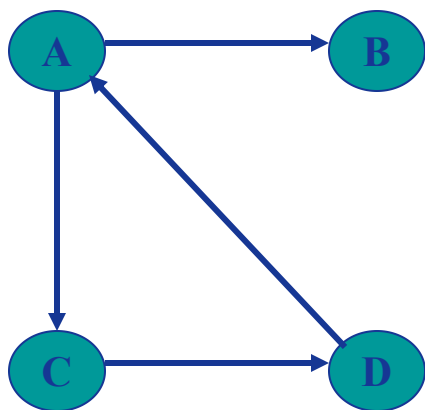
若 G 是一个非连通无向图，共有28条边，则该图至少有9个顶点。

强连通图 有向图中，任何一对顶点间都存在路径。

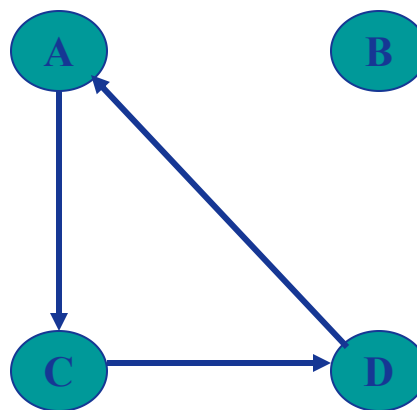
强连通分量 有向图中的极大连通子图，极大是指顶点数。

思考：要连通具有 $n(n>1)$ 个顶点的有向图，至少需要____条边

有向图G



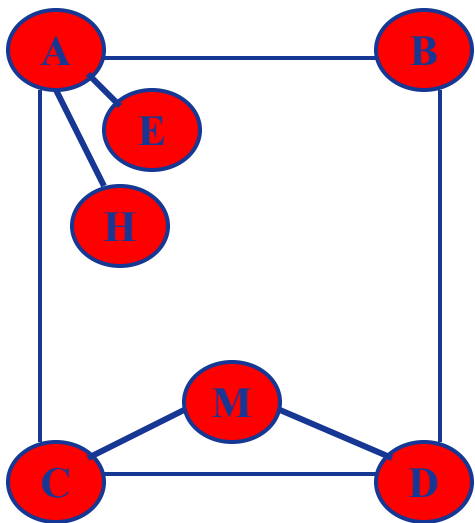
有向图G的两个强连通分量



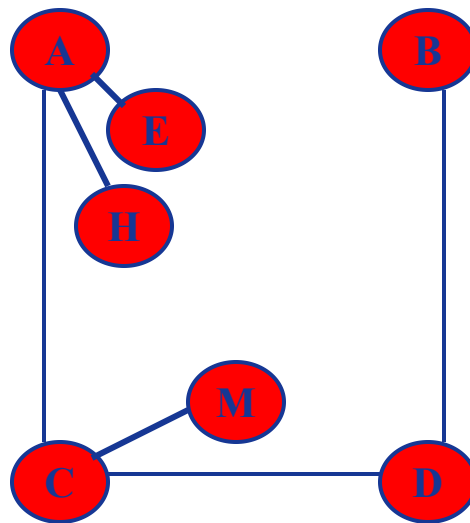
思考：一个有 n 个结点的有向图，最少有（ 1 ）个强连通分量，
最多有（ n ）个强连通分量。

生成树 包含图中全部顶点的极小连通子图，极小是指边数。

无向图G



无向图G的生成树



包含图的所有 n 个结点，
但只含图的 $n-1$ 条边。
在生成树中添加一条边之后，
必定会形成回路或环。

有向树 图中恰有一个顶点入度为0，其余顶点入度均为1。

生成森林 有向图中，包含所有顶点的若干棵有向树构成的子图。

7.2 图的存储结构

1. 数组+邻接矩阵/加权邻接矩阵(顺序存储方式)

- 无权值的图的邻接矩阵

无权值图有 n 个顶点，可用 $n * n$ 的布尔矩阵 A 表示该图。

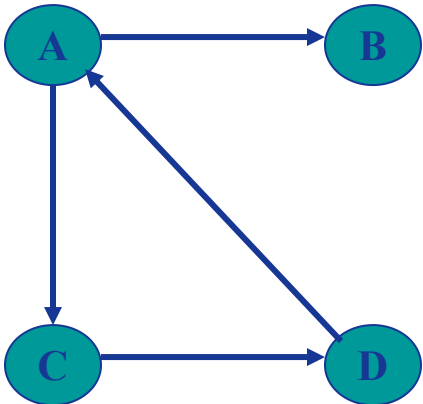
$A[i,j] = 1$ ，表示有边 (i,j) 或者 $\langle i,j \rangle$ ；

$A[i,j] = 0$ ，表示没有边 (i,j) 或者 $\langle i,j \rangle$ 。

$A[i,i] = 0$ ，表示顶点 i 没有自环边。

i 结点的度: 对无向图而言，为 i 行或 i 列之和；对有向图 i 行之和为出度， i 列之和为入度。

对无向图必然为对称矩阵，可压缩为上三角矩阵或下三角矩阵。

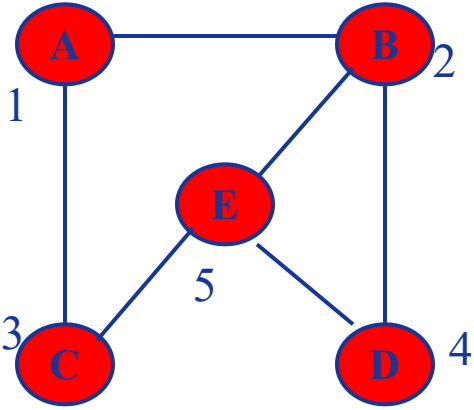


表示成右图矩阵

$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

顶点数组

1	A
2	B
3	C
4	D



表示成右图矩阵

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{pmatrix}$$

1	A
2	B
3	C
4	D
5	E

思考：n个顶点的连通图用邻接矩阵表示时,该矩阵至少有 2(n-1) 个非零元素。
n个顶点的强连通图用邻接矩阵表示时,该矩阵至少有 n 个非零元素。

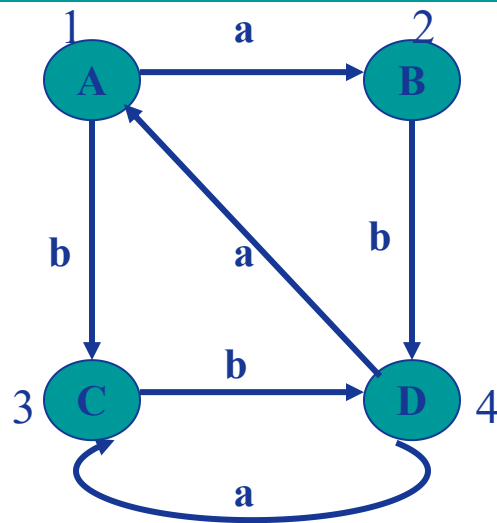
- 带权图的加权邻接矩阵

$A[i,j] = w_{ij}$, 表示有边 (i,j) 或者 $\langle i,j \rangle$, 权值为 w_{ij} ;

$A[i,j] = \infty$, 表示没有边 (i,j) 或者 $\langle i,j \rangle$ 。

$A[i,i] = 0$, 表示顶点 i 没有自环边。

无向图仍然是对称矩阵, 可压缩存储。



表示成右图矩阵

$$\begin{bmatrix} & a & b & \\ & & & b \\ a & & & \\ & & a & \end{bmatrix}$$

优点: 判断任意两点之间是否有边方便, 仅耗费 $O(1)$ 时间。

缺点: 即使 $\ll n^2$ 条边, 也需至少内存 $n^2/2$ 单元, 太多; 仅读入数据需 $O(n^2)$ 时间, 太长。

[邻接矩阵结构]

```
#define MAXV <最大顶点数>
typedef struct
{
    int no; //顶点编号
    elemtype data; //顶点值
} VertexType;
typedef struct
{
    int edges[MAXV][MAXV]; //邻接矩阵数组
    int n,e; //顶点数和边数
    VertexType vexs[MAXV]; //顶点数组
} Mgraph;
```

2. 邻接表 顺序存储+链式存储

- 用顶点表、边表表示有向图或无向图。
- 顶点表：用数组的形式存放所有的顶点。
- 边表（出边表）：从某个顶点出发的边结点组成的单链表。
- **优点**：内存消耗 = 顶点数 + 边数
- **缺点**：确定 $i \rightarrow j$ 是否有边，最坏需耗费 $O(n)$ 时间。无向图同一条边表示两次边表空间浪费一倍。有向图中寻找进入某结点的边，非常困难。

顶点表中的结点的表示：

- 用数组

data	firstarc
------	----------

data:顶点的数据域，保存顶点的数据元素的值。

firstarc:顶点的指针域，指向自该顶点出发的第一条边的边结点地址。

(出) 边表中的边结点的表示：

weight	adjvex	nextarc
--------	--------	---------

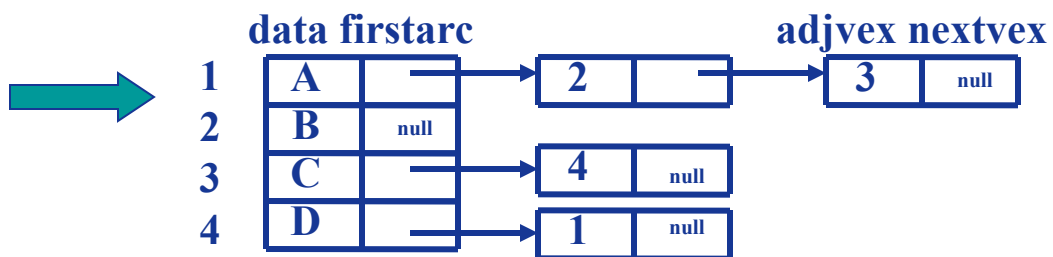
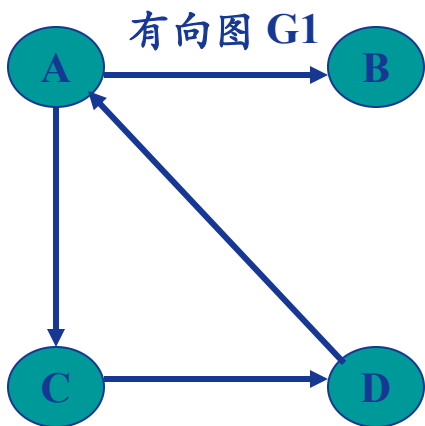
weight:保存边的权值，**可选项**。

adjvex:边结点的指针，给出本条边依附的另一顶点（非出发顶点）的编号。

nextarc:顶点的指针，给出自同一顶点出发的下一条边的边结点的地址。

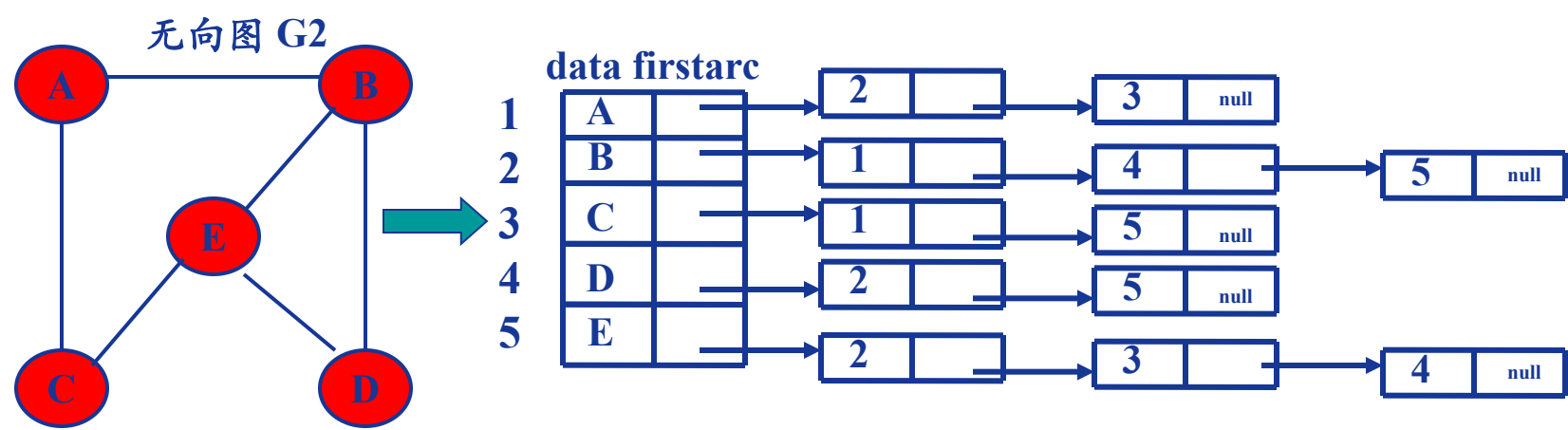
[邻接表的存储结构]

```
typedef struct Anode //边结点的类型
{
    int adjvex; //边的终点
    struct Anode *nextarc; //同一顶点出发的下一条边
    int weight; //边的权值 可选项
} ArcNode;
typedef struct Vnode //顶点类型
{
    elemtype data; //顶点的值
    ArcNode *firstarc; //指向该顶点出发的第一条边
} Vnode;
typedef Vnode AdjList[MAXV]; //AdjList是顶点表类型
typedef struct
{
    AdjList adjlist; //顶点表
    int n,e; //顶点数和边数
} Agraph;
```



寻找某个顶点的入边非常困难!!!

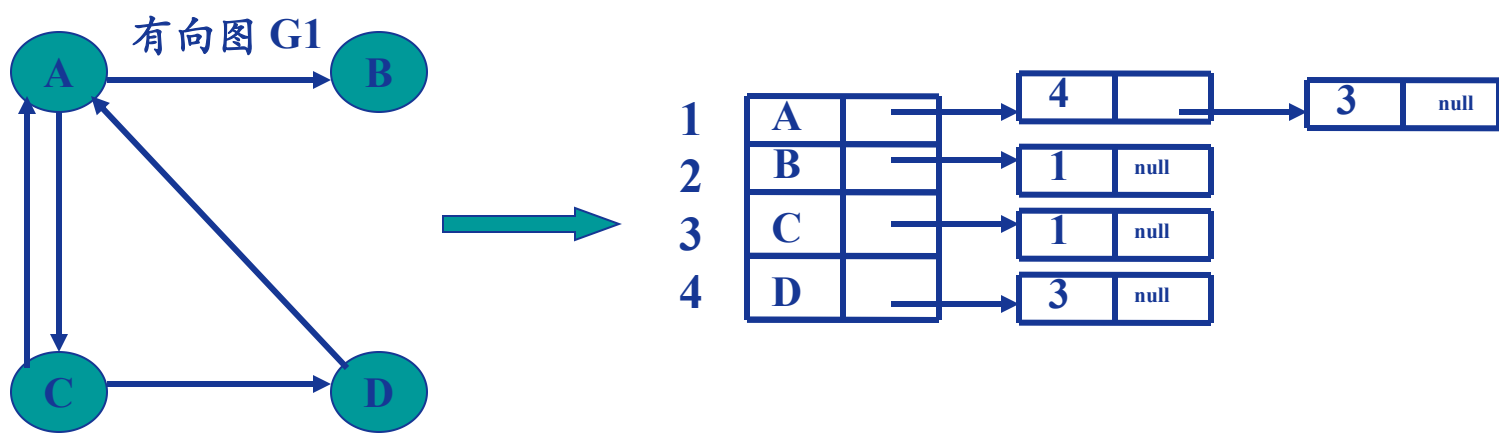
改进: 建立逆邻接表或十字链表



六条边却用了 12 个边结点!!!

改进：建立邻接多重表

•逆邻接表：边表采用入边表



[示例] 邻接矩阵存储的图的生成算法

由数组A[][]生成邻接矩阵g, **INF**表示 ∞

```
void createMat(Mgraph &g, int
A[][MAXV], int n)
{
    g.n = n;
    g.e = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
        {
            g.edges[i][j] = A[i][j];
            if (A[i][j] != 0 && A[i][j] != INF)
                g.e++;
        }
}
```

[示例] 邻接表存储的图的生成算法

由数组A[][]生成邻接表G

```
void createAdj(Agraph &*G, int A[][MAXV], int n)
{
    ArcNode *p;
    G = (Agraph *)malloc(sizeof(Agraph))
    G->n = n; G->e = 0;
    for (int i = 0; i < n; i++)
        G->adjlist[i].firstarc = NULL;
    for (i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (A[i][j] != 0 && A[i][j] != INF) \存在边
            {
                p = (ArcNode *)malloc(sizeof(ArcNode));
                p->adjvex = j;
                p->weight = A[i][j];
                p->nextarc = G->adjlist[i].firstarc; //头插法;
                G->adjlist[i].firstarc = p;
                G->e++;
            }
}
```

[示例] 将邻接表存储的带权图转换成邻接矩阵

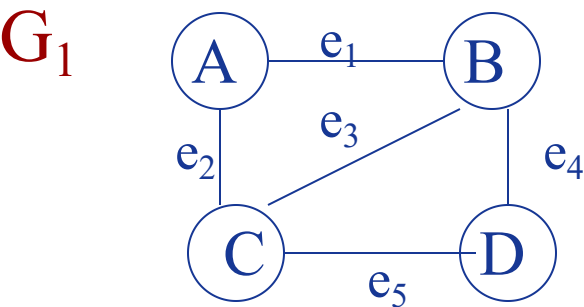
```
void convert(Agraph *G, Mgraph &g)
{
    int i, j; ArcNode *p ;
    g.n = G->n; g.e = G->e;
    for (i =0; i<g.n, i++)
        for (j =0; j<g.n, j++)
            g.edges[i][j] = (i ==j)?0 : INF; //邻接矩阵赋初值
    for (i =0; i<g.n, i++) //遍历每个顶点
    {
        p =G->adjlist[i].firstarc;
        while(p!= NULL) // //遍历每个邻接顶点
        {
            g.edges[i][p->adjvex] = p->weight;
            p = p->nextarc; //找下一个邻接顶点
        }
    }
}
```

[两种常用存储结构的比较]

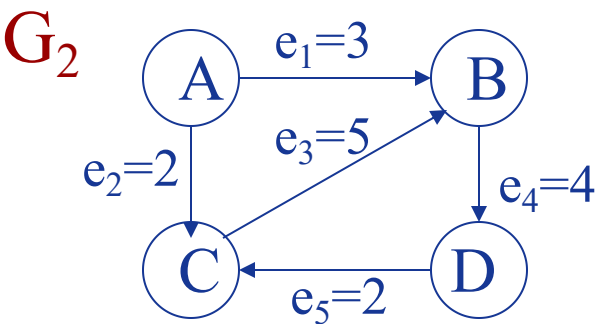
邻接矩阵			邻接表	
无向图 有向图			无向图 有向图	
空间	顶点表结点	n n	顶点表结点	n n
	邻接矩阵	$n^2/2$ n^2	边表结点	$2e$ e
适合稠密图			适合稀疏图	
时间	求顶点的度	扫描行/列 $O(n)$	扫描某个边表 最坏 $O(n)$	求出度同 无向图; 求入度 $(n+e)$
	判定边是 否存在	$O(1)$	扫描某个边表 最坏 $O(n)$	
	求边的数目	检测矩阵 $O(n^2)$	检测每个边表 $O(n+2e)$	

3. 图的其它存储结构

(1) 关联矩阵 顺序存储



	e_1	e_2	e_3	e_4	e_5
A	1	1	0	0	0
B	1	0	1	1	0
C	0	1	1	0	1
D	0	0	0	1	1

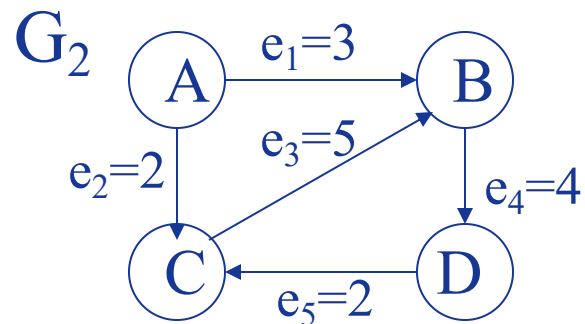


	e_1	e_2	e_3	e_4	e_5
A	-3	-2	0	0	0
B	3	0	5	-4	0
C	0	2	-5	0	2
D	0	0	0	4	-2

(2) 十字链表 顺序存储+链式存储

适用对象：有向图。

[例]

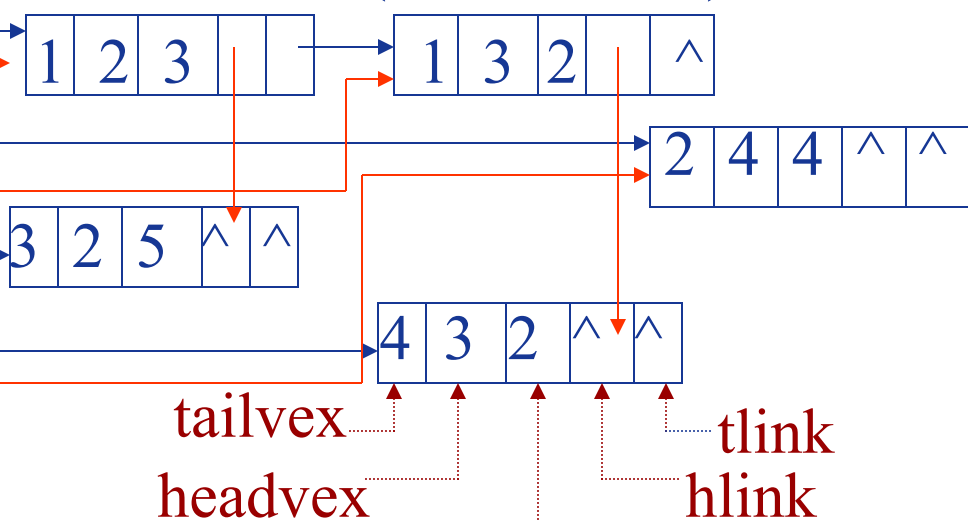


顶点线性表

1	A	^	
2	B		
3	C		
4	D		

data firstout firstin

弧的十字链表(弧头、弧尾)



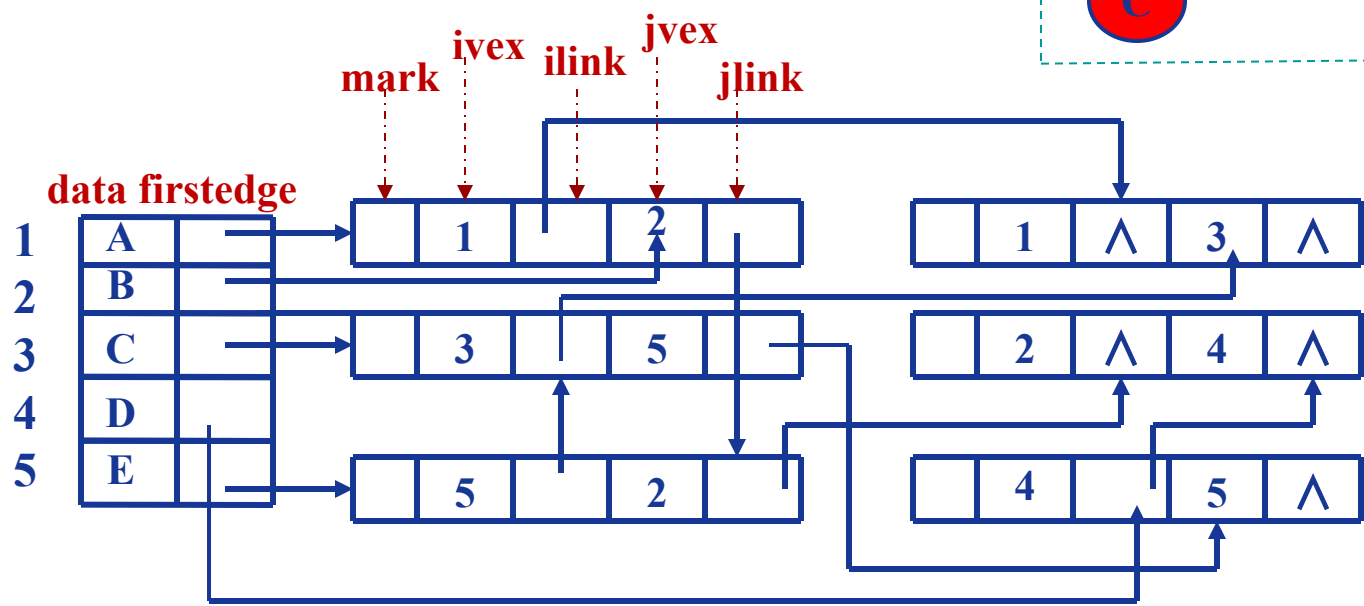
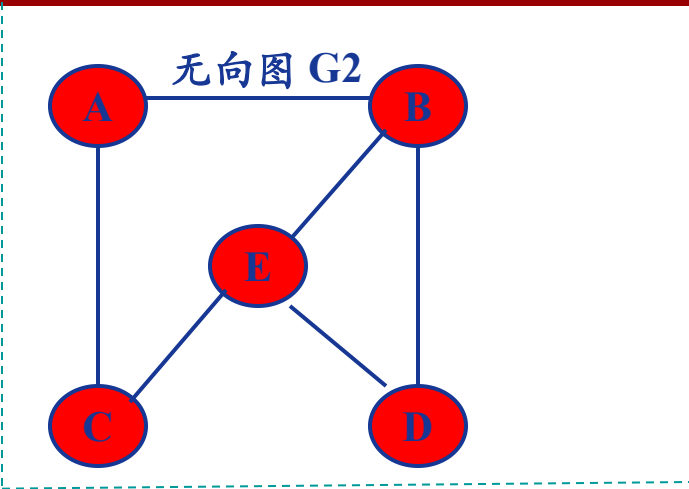
查询入边和出边比较容易!

(3) 邻接多重表 顺序存储+链式存储

适用对象：无向图。
特点：方便无向图中对边的操作。

[例]

顶点线性表 邻接多重表



边表中的边结点只需存放一次，节约内存。

7.3 图的遍历

BUPT

两种遍历方式

- ❖ 深度优先遍历 (树的先根遍历的推广) Depth-First Search
- ❖ 广度优先遍历 (树的按层次遍历的推广) Breadth-First Search

1. 深度优先遍历

注意：每个顶点只能被访问一次，因为一个顶点可以和其它的任何顶点相邻接，为了避免对一个顶点的重复访问，必须对访问过的顶点加以标记。顶点的邻接顶点的次序是任意的，因此深度优先搜索的序列可能有多种。

[遍历过程]

- ❖ 1、选中第一个被访问的顶点s。
- ❖ 2、对顶点s作已访问过的标志。
- ❖ 3、依次从顶点s的未被访问过、邻接顶点出发，进行深度优先遍历。转向2。
- ❖ 4、如果还有顶点未被访问，则选中一个起始顶点，转向2。
- ❖ 5、所有的顶点都被访问到，则结束。

深度优先的含义：只要可能就尽量深入，深度优先搜索总是对最近才发现的结点v的出边进行搜索，直到所有出边都被搜索为止。一旦v所有出边都被发现，搜索则回溯到v的前驱结点，来搜索该前驱结点的出边。

数据结构：主——图的存储结构

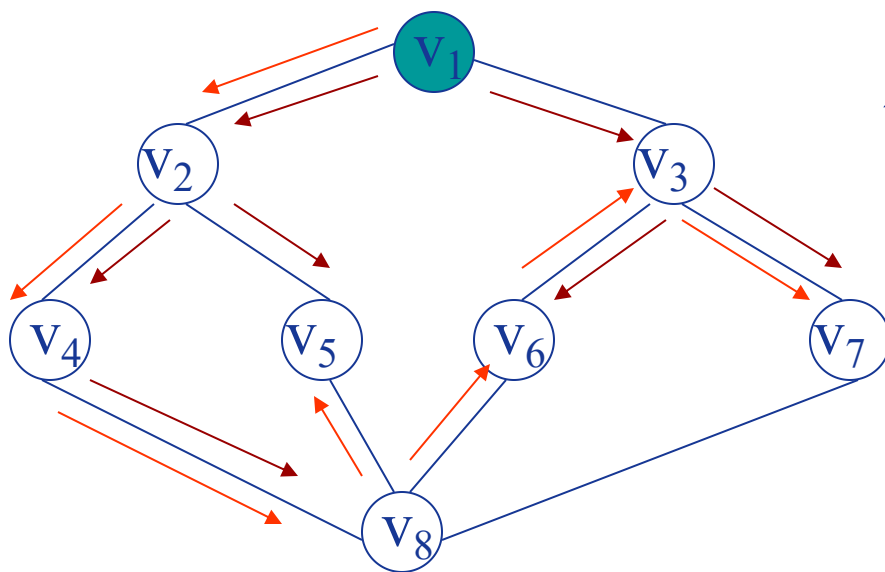
辅——数组 `visited[1..n]` //用于标识结点是否已被访问过

[例]无向图的深度优先遍历

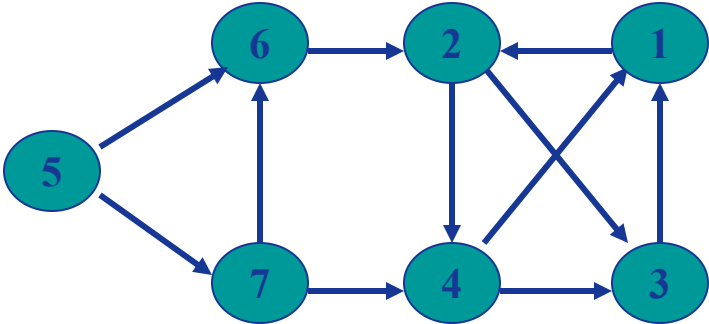
设从 v_1 出发遍历

深度： $v_1 v_2 v_4 v_8 v_5 v_6 v_3 v_7$

广度： $v_1 v_2 v_3 v_4 v_5 v_6 v_7 v_8$



[例] 有向图的深度优先遍历



从顶点 1 出发的搜索序列:

1、2、3、4; 没有搜索

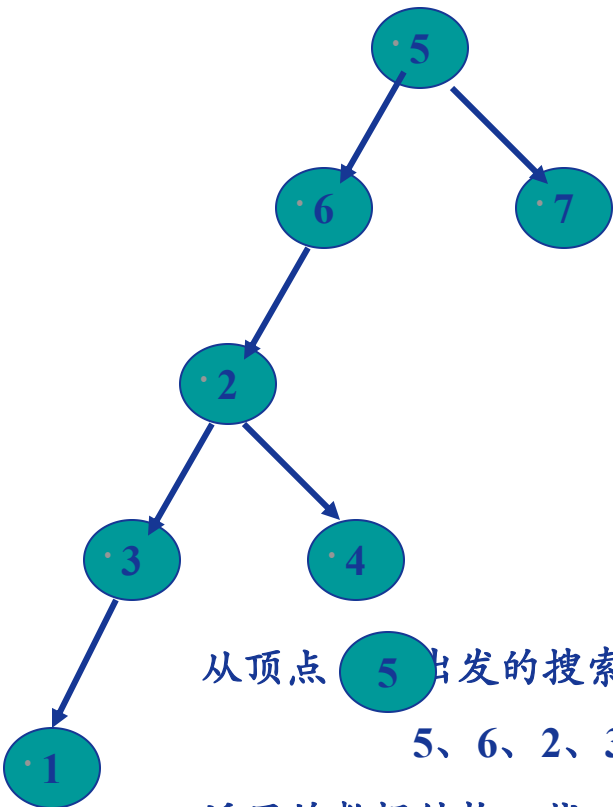
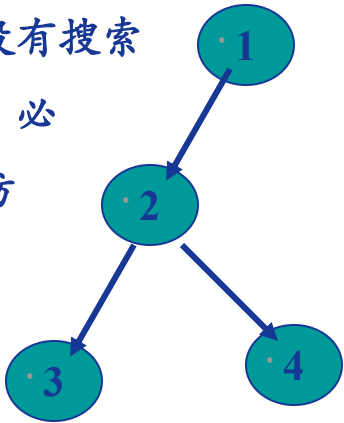
到所有的顶点, 必

须另选图中未访

问过的结点,

继续进行

搜索。



从顶点 5 出发的搜索序列:

5、6、2、3、1、4、7

适用的数据结构: 栈

DFS算法说明

[染色]

为跟踪DFS算法进展，利用全局数组Color为顶点染色。

`typedef enum Vclolor{white, gray, black} vextexColor; vextexColor color[MAXV];` `color[v]=white`，表示顶点v染白色；`Color[v]=gray`，表示顶点v染灰色；`Color[v]=black`，表示顶点v染黑色；

①DFS初始化时所用顶点都染白色。

②灰色或黑色表示顶点已被发现。算法过程中第一次遇到一个顶点，就称之为被发现（被访问），该顶点颜色从白色变为灰色。

③若一个顶点被发现后，其邻接表/邻接矩阵所在行被扫描完，则顶点颜色由灰色变为黑色。

[打时间戳]

在DFS运行过程中利用全局数组b[]和f[]为每个顶点打上两个时间戳。

`int b[MAXV]; int f[MAXV];`

①第1个时间戳：b[v]记录顶点v第一次被发现（被访问）的时间，即染成灰色的时间。

②第2个时间戳：f[v]记录顶点v的邻接表/邻接矩阵所在行被扫描完的时间，即被染黑色的时间。

③对于任何一个结点v而言，只能有一个发现时间b[v]和一个完成时间f[v]，且 $b[v] < f[v]$ 。

[前驱顶点]

在DFS运行过程中利用全局数组p[]为每个顶点记录其在深度优先搜索中的前驱顶点；

`int p[MAXV];` 在扫描已发现顶点u（灰色顶点）的邻接顶点时，每发现一个白色顶点v，就置 `p[v]=u`；这表示顶点u是顶点v在深度优先搜索中的前驱顶点。

[算法实现]

(1)访问顶点v，并记录v已被访问

(2)依次从v的未访问的邻接点出发，深度优先搜索图G。

```
bool visited[MAXV]; //访问标识
int time; //全局变量，用于打时间戳
void DFSTraverse( Graph G, Status ( * VisitFunc) (int v))
{
    for ( v=0; v <G.vexnum; ++v )
    {
        visited[v] = FALSE;
        color[v]=white; //初始染白色
        p[v]=-1;
    }
    time =0; //time初值置为0
    for ( v=0; v <G.vexnum; ++v )
        if (!visit[v] )
            DFS(G,V);
}
```

```
void DFS( Graph G, int v )
{
    visited[v] = TRUE;
    PrintData(v);
    color[v]=gray; //访问时染灰色
    time++;
    b[v] = time; //开始时间
    for ( w = FirstAdjVex(G, v) ; w ;
        w = NextAdjVex (G, v, w) //有(v,w)<<v,w>
        if (!visit[w]) //或颜色为白色时
        {
            p[w]= v; DFS(G,W);
        }
    color[v]=black; //for循环退出后染黑色
    time++;
    f[v] = time; //结束时间
}
```

[边的分类]

深度优先搜索边的性质：通过深度优先搜索可以对输入图 $G=(V,E)$ 的边进行分类。利用对图得到的深度优先森林，可以定义四种边。

[深度优先遍历中的四种边]

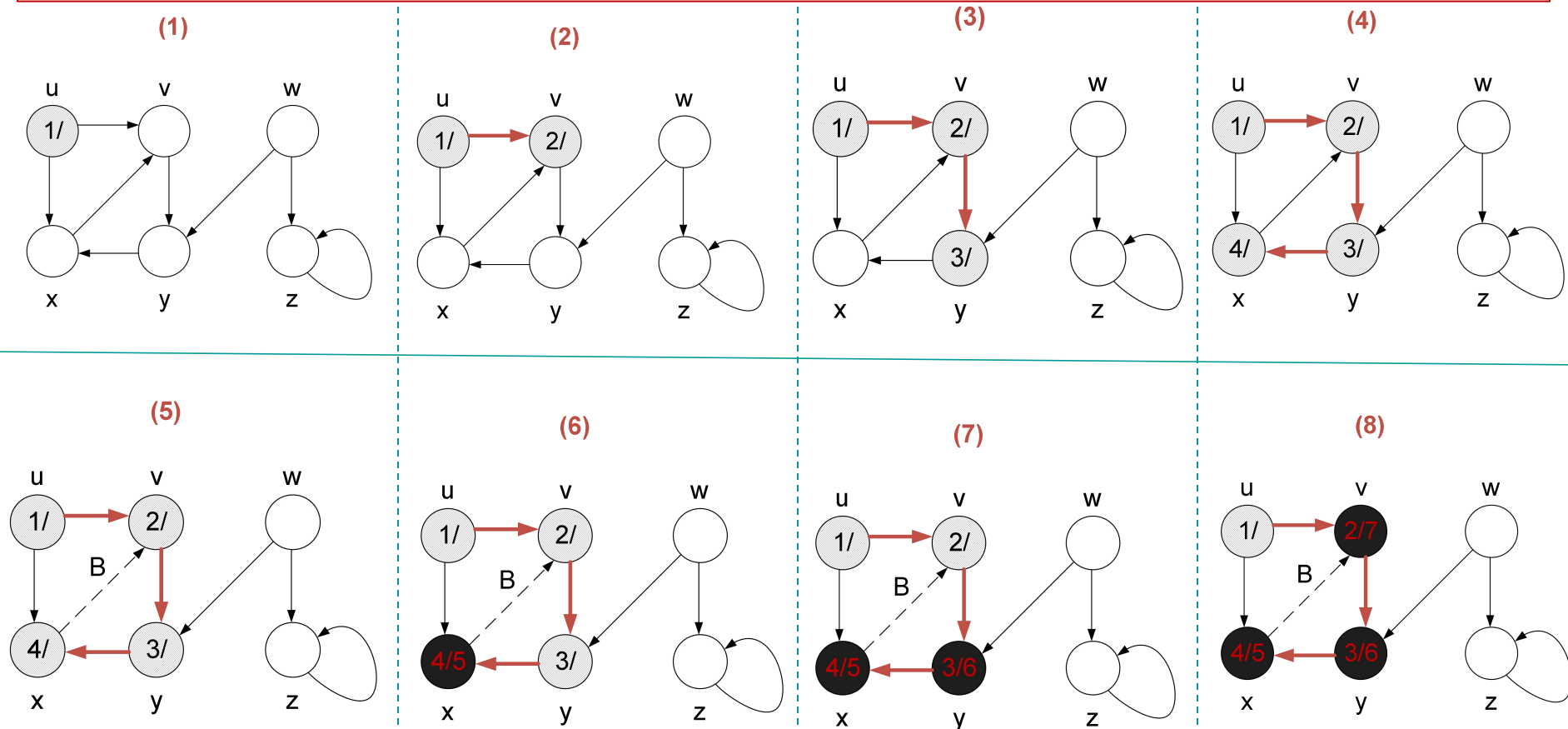
- ① **树边**：为 G_p 的边。如果结点 v 是因为算法对 $(u,v)/<u,v>$ 的探索而首次被发现的，即执行语句 $p[v]=u$ ，则 $(u,v)/<u,v>$ 是一条树边（连接父子）。
- ② **后向边**：后向边 $(u,v)/<u,v>$ 是将 u 连接到其在深度优先树 G_p 中祖先顶点 v 的边（连接子父或孙祖）。
- ③ **前向边**：是将 u 连接到 G_p 中一个后代 v 的边 $(u,v)/<u,v>$ （连接祖孙）。
- ④ **横向边**：其他边。可以是连接同一棵深度优先树 G_p 中的结点，只要一个结点不是另一个结点的祖先（即连接兄弟结点），或者也可以是连接不同深度优先树 G_p 中的两个结点。只针对于有向图

定理：对无向图 G 进行深度优先搜索，每条边要么是树边，要么是后向边。

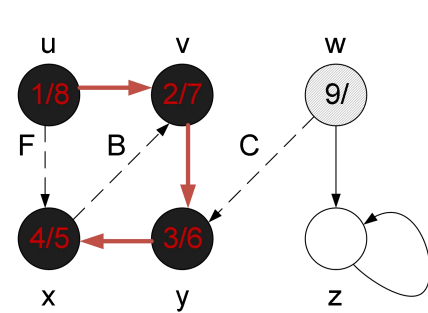
DFS算法实例

说明1: 红色边表示该边为树边; 边变为虚线, 表示边为非树边, 其中: B表示后向边, F表示前向边, C表示横向边。

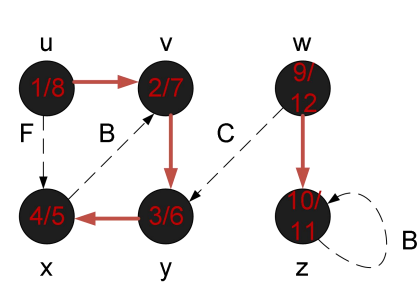
说明2: 随着DFS的运行, 动态地为顶点染色, 并标注了时间戳, 格式: $b[v]/f[v]$ -开始时间/结束时间。



(12)



(16)



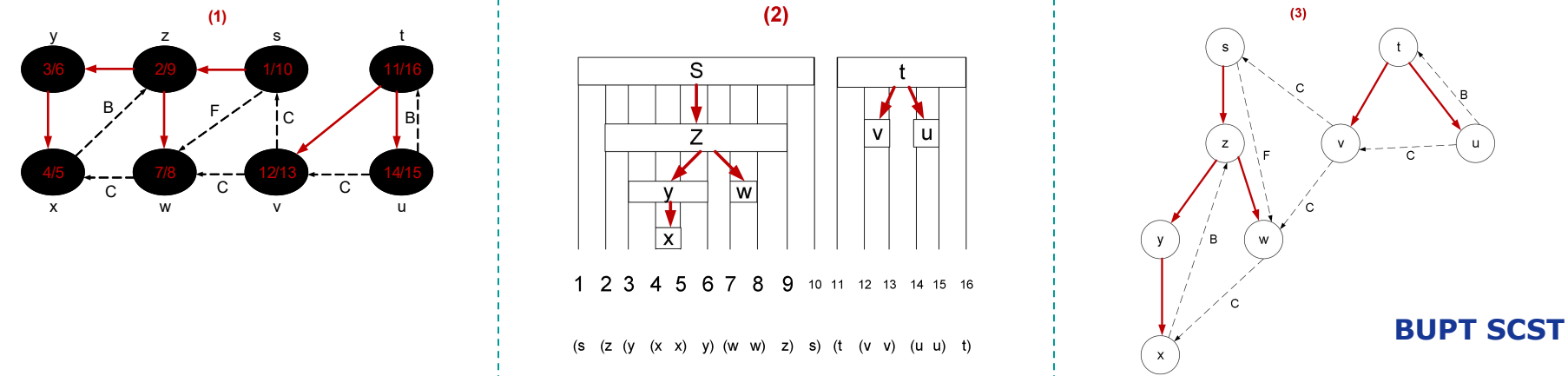
[深度优先搜索的性质]

定义1: 对于图G和源顶点s, **深度优先搜索的前驱子图**是 $G_p=(V_p,E_p)$, 其中 $V_p=\{v \in V, p[v] \neq -1\} \cup \{s\}$, $E_p=\{(p[v], v): v \in V_p - \{s\}\}$.

性质1: 图G的深度优先前驱子图, 形成了一个由1棵或多颗树组成的森林。当且仅当DFS(G,v)在算法对顶点u的邻接表/矩阵搜索时被调用, $u=p(v)$ 。此外, 当且仅当顶点v在顶点u是灰色时段被发现, 顶点v是深度优先森林中顶点u的后代,

括号化结构: 如果以左括号“(u”表示顶点u被发现的时间, 右括号“)u”表示顶点u完成时间。则发现和完成的历史记载形成一个规整的表达式。所谓规整是指: 所有括号都适当的嵌套在一起 (即括号是匹配的)。

示例: 图(1)有向图深度优先搜索的结果, 顶点标注了时间戳, 边标注了类型: 红色树边, 前向边F, 后向边B, 横向边C。
图(2)每个顶点发现时间和完成时间对应的括号化结构, 每个矩形横跨相应结点的发现和完成时间。(2)只显示树边, 非树边省略。
图(3)对图(1)进行重画, 红色树边和所有顶点构成了深度优先森林中的两棵树。



括号化定理：在对有向图和无向图 $G=(V,E)$ 进行的任意深度优先搜索中，对于任意两个顶点 u 和 v 而言，下面三种情况之一成立：

- (1) 时间区间 $(b[u], f[u])$ 与时间区间 $(b[v], f[v])$ 完全分离，在深度优先森林中，顶点 u 不是顶点 v 的后代，顶点 v 也不是顶点 u 的后代。
- (2) 时间区间 $(b[u], f[u])$ 完全包含在时间区间 $(b[v], f[v])$ 之内，在深度优先森林中，顶点 u 是顶点 v 的后代。
- (3) 时间区间 $(b[v], f[v])$ 完全包含在时间区间 $(b[u], f[u])$ 之内，在深度优先森林中，顶点 v 是顶点 u 的后代。

白色路径定理：在有向图和无向图 $G=(V,E)$ 的深度优先森林中，顶点 v 是顶点 u 的后代，当且仅当在发现顶点 u 的时间 $b[u]$ ，存在一条从顶点 u 到顶点 v 的全部由白色顶点所构成的路径。

顶点变灰色\ $b[v]$ 赋值\ “(v” $\longrightarrow$ 入栈
 顶点变黑色\ $f[v]$ 赋值\ “v)” $\longrightarrow$ 出栈

[DFS的非递归实现]

- (1)访问一个顶点 v ，并记录它已被访问；
将它的所有未访问的邻接顶点入栈；
- (2)如果栈空，则退出；
否则，栈中一顶点出栈；
- (3)如果该顶点已被访问过，则转(2)
否则，转(1)

采用邻接表存储，DFS算法的实现

[递归算法实现]

```
Void DFS(Agraph *G, int v)
{
    ArcNode p;
    visit[v] = TRUE;

    printf("%3d", v);
    p = G->adjlist[v].firstarc;
    while(p != NULL)
    {
        if (visit[p->adjvex] == FALSE)
            DFS(G, p->adjvex);
        p = p->nextarc;
    }
}
```

[非递归算法实现]

```
void DFS(Agraph *G, int v)
{
    int St[MAXV]; int top = -1, w, x;
    ArcNode p;
    visit[v] = TRUE; printf("%3d", v);
    top++; St[top] = v;
    while (top > -1)
    {
        x = St[top]; //取栈顶顶点x
        p = G->adjlist[x].firstarc; //找顶点x的一个邻接顶点
        while (p != NULL)
        {
            w = p->adjvex; //x的邻接顶点记为w
            if (visit[w] == FALSE) //若w未被访问
            {
                printf("%3d", w); //访问w
                visit[w] = TRUE;
                top++; St[top] = w; //w入栈
                break; //跳出循环，处理栈顶的顶点（先进先出）
            }
            p = p->nextarc; //取顶点x的下一个邻接顶点
        }
        if (p == NULL) top--; //x无未被访问的邻接顶点，退栈
    }
}
```

采用邻接矩阵存储，DFS算法的实现

邻接矩阵实现DFS

```
Void DFS(Mgraph g, int v)
```

```
{
```

```
    visit[v] = TRUE;
```

```
    printf("%3d", v);
```

```
    for (int i=0; i<MAXV; i++)
```

```
        if(g.edges[v][i] != 0 && if(g.edges[v][i] != INF && visited[i] == FALSE )
```

```
            DFS(g,i);
```

```
}
```

不包括染色、打时间戳和记录前驱等功能，这些功能可依据需要自行实现

[算法时间复杂度分析]

主要操作：查找每个顶点的所有邻接点

❖ 邻接矩阵 $O(n^2)$

❖ 邻接表 无向图 $O(n+2e)$

有向图 $O(n+e)$

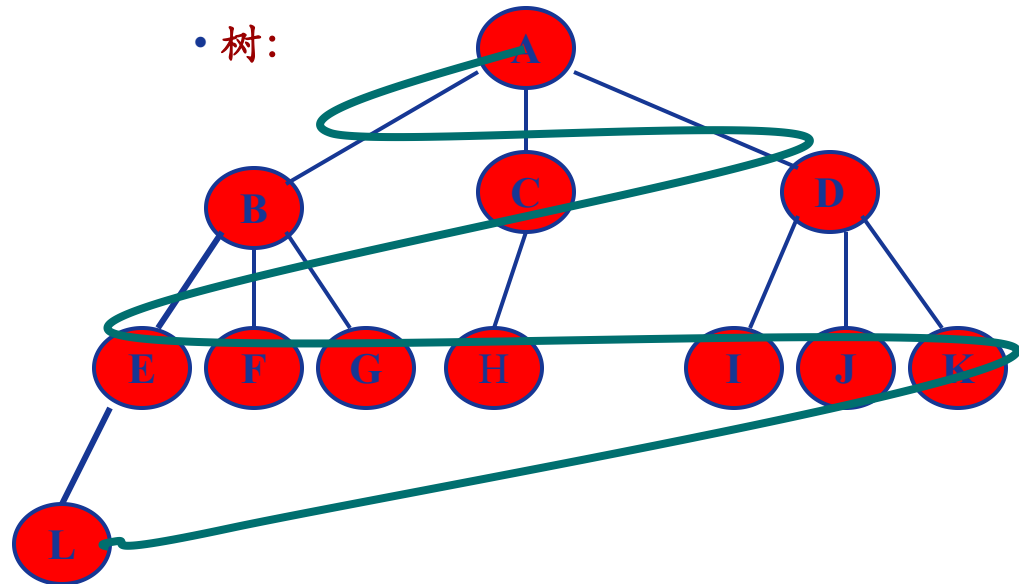
2. 广度优先遍历(BFS)

[遍历过程]

- 1、选中第一个被访问的源顶点 s
- 2、对源顶点 s 作已访问过的标志。
- 3、依次从源顶点 s 的未被访问过的第一个、第二个、第三个.....第 m 个邻接顶点 w_1 、 w_2 、 w_3 w_m ，且进行标记。
- 4、依次访问顶点 w_1 、 w_2 、 w_3 w_m 的邻接顶点，且进行标记。
- 5、如果还有顶点未被访问，则选中一个起始顶点，也标记为 s ，转向2。
- 6、所有的顶点都被访问到，则结束。

广度优先的含义：BFS算法始终将已发现的顶点和未发现的顶点之间的边界，沿其广度方向向外扩展。即算法需要先访问到距离源顶点 s 为 k 的所有顶点之后，才会访问到距离源顶点 s 为 $k+1$ 的顶点。
广度优先算法是很多算法的原型，Prim和Dijkstra算法都采用了类似广度优先的思想。

• 树:



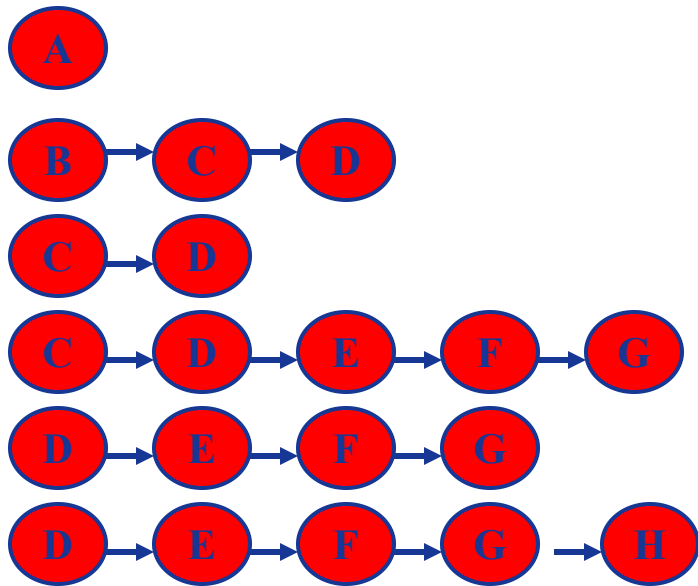
• 队列的变化:

- 1、A进队
- 2、A出队、访问，队空
- 3、A的儿子结点进队
- 4、B出队、访问
- 5、B的儿子结点进队
- 6、C出队、访问
- 7、C的儿子结点进队
-

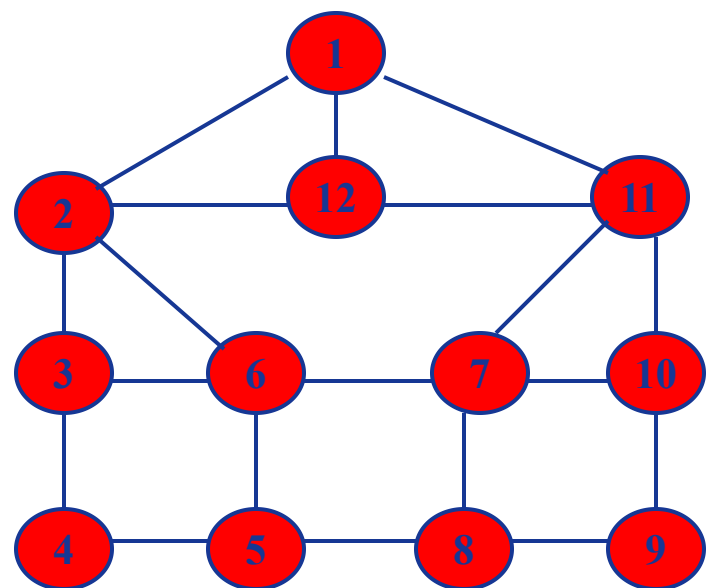
树的按层次进行访问的次序:

A、B、C、D、E、F、G、H、
I、J、K、L

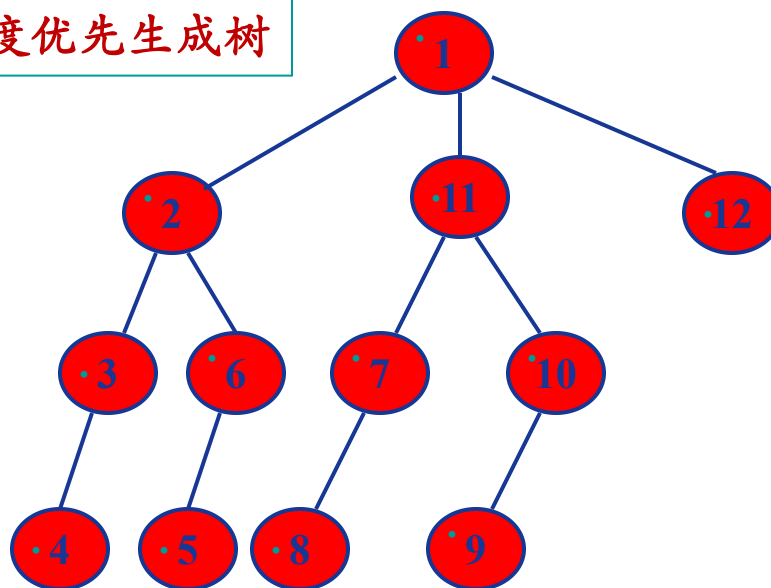
适用的数据结构: 队列



- 无向图的实例：为了说明问题，邻接顶点的访问次序以序号为准。序号小的先访问。如：
顶点 1 的邻接顶点有三个 2、12、11，则先访问结点 2、11，再访问结点 12。



广度优先生成树



图的广度优先的访问次序：

1、2、11、12、3、6、7、10、4、5、8、9

适用的数据结构：队列

[BFS算法思想]

(1)访问源顶点s，并记录它已被访问；

顶点s入队列Qu；

(2)如果队列Qu空，则退出；

否则，从队中取出一顶点；

(3)求该顶点的一个邻接点；

如果此邻接点未被访问，则访问它，并记录它已被访问，将其入队列Qu；

(4)如果该顶点还有下一个邻接点，则转(3)；

否则，转(2)

为跟踪BFS算法进展，假设在BFS运行过程中为顶点染色。

①BFS初始化时所用顶点都染成白色。

②灰色或黑色表示顶点已被发现。算法过程中第一次遇到一个顶点（入队前），就称之为被发现，该顶点颜色从白色变为灰色。

③若边 $(u,v) \in E$ ，且u是黑色，则v可能是灰色或黑色；即顶点u为黑色，表示u所有邻接顶点都被发现（出队后）。

④所有灰色顶点表示已知和未知的边界，灰色顶点必在队列Qu中。

假设在BFS算法运行过程通过三个全局数组中每个结点维护三个属性：

①typedef enum Vclolor{white, gray, black} vextexColor; **vextexColor color[MAXV]**; color[v]=white, 表示顶点v涂白色； Color[v]=gray, 表示顶点v涂灰色； Color[v]=black, 表示顶点v涂黑色；

②**int p[MAXV]**; 在扫描已发现顶点u（灰色顶点）的邻接顶点时，每发现一个白色顶点v，就置p[v]=u；这表示顶点u是顶点v在广度优先搜索中的前驱顶点；

③ **int d[MAXV]**; d[v]记录通过BFS算法计算出的从源顶点s出发到顶点v的距离。

[算法] 邻接表存储的图的广度优先遍历

```

Void BFS(Agraph *G, int s) {
    ArcNode *q;
    int Qu[MAXV], front=0, rear=0;
    int w;
    for(int i=0; i<MAXV; i++) { //d[i]初值为无穷，表示目前未确定s到顶点i的路径,p[i]初值为-1表示顶点i无前驱，
        visit[i] = FALSE; d[i]=INF; p[i]= -1; color[i] = white;
    }
    printf("3d%",s); visit[s]=TRUE; d[s]=0; color[w]=gray;
    rear = (rear+1)%MAXV; QU[rear] =s;//源顶点s入队
    while(front != rear) { //队列不为空
        front =(front +1)%MAXV; w=QU[front]; //队头顶点w出队
        q = G->adjlist[w].firstarc; //找到w的第一条出边（第一个邻接顶点）；
        while(q!=NULL) {
            if(visit[q->firstarc] == FALSE) { //未被访问
                printf("2d%",q->adjvex); visit[q->adjvex] = TRUE; //访问w的邻接顶点
                d[q->adjvex] = d[w]+1; p[q->adjvex] = w; color[q->adjvex] = gray;
                rear = (rear+1)%MAXV; Qu[rear] = q->adjvex; //入队
            }
            q = q->nextarc;
        } // end while(q!=NULL)
        color[w]=black;
    } //end while(front != rear)
}

```

[算法] 邻接矩阵存储的图的广度优先遍历

```

void BFS(Mraph g, int s)
{
    int Qu[MAXV], front=0, rear=0;
    bool visit[MAXV]; int p[MAXV];
    int d[MAXV]; // 用于记录利用BFS算法得到的从s到其他顶点的距离
    int w; int i;
    for(i=0; i<MAXV; i++) {visit[i] = FALSE; d[i]=INF; p[i]=-1; color[i] = white;}
    printf("3d%",s); visit[s]=TRUE; d[s] = 0; color[i] = gray;
    rear = (rear+1)%MAXV; QU[rear] =s;//源顶点s入队
    while (front != rear) //队列不为空
    {
        front =(front +1)%MAXV; w=QU[front]; //队头顶点w出队
        for (i=0; i<MAXV; i++)
            if(g.edges[w][i] != 0 && if(g.edges[w][i] != INF && visited[i] == FALSE )
            {
                printf("3d%",i);
                visit[i] = TRUE;
                d[i]=d[w]+1; p[i] = w; color[i]=gray;
                rear = (rear+1)%MAXV; Qu[rear] = i; //入队
            }
        color[i] = black;
    }
}

```

[时间复杂度分析]

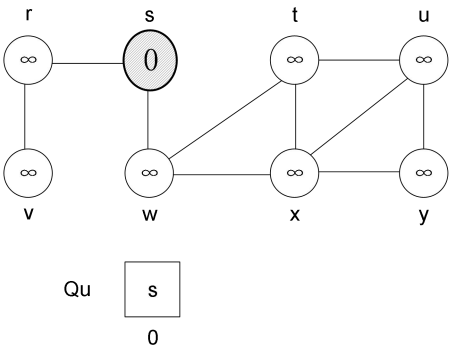
主要操作：查找每个顶点的所有邻接点

邻接矩阵 $O(n^2)$

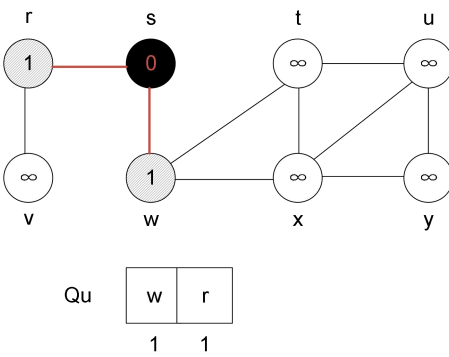
邻接表 无向图 $O(n+2e)$

有向图 $O(n+e)$

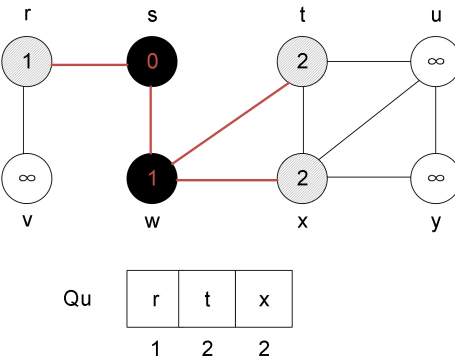
BFS实例 ①



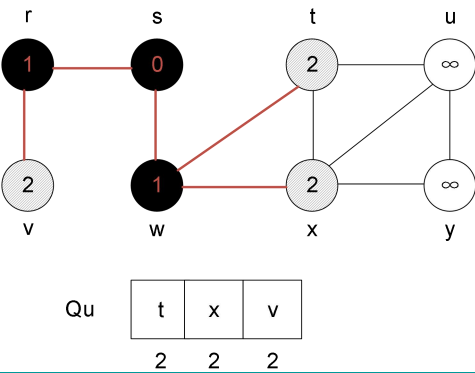
②



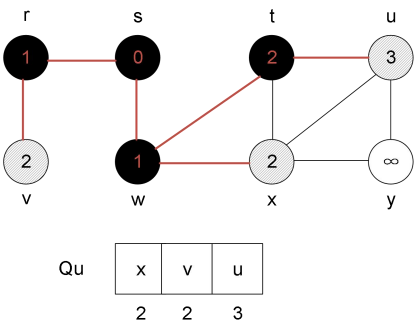
③



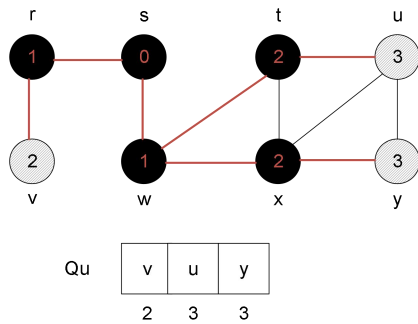
④



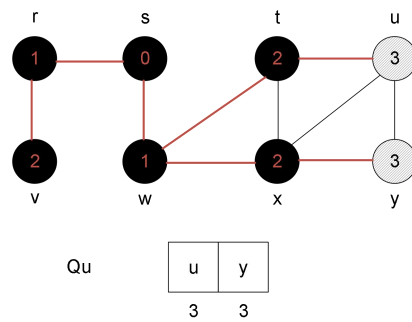
⑤



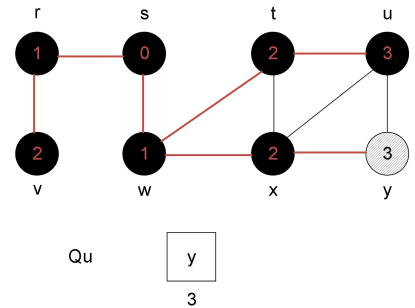
⑥



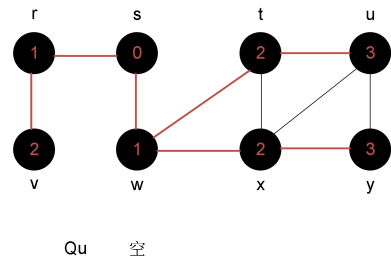
(7)



(8)



(9)

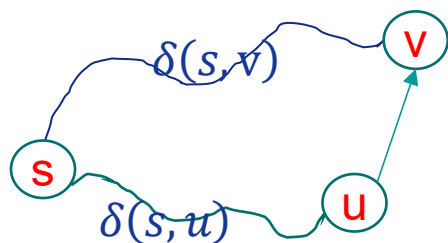


[广度优先的性质]

最短路径的定义：对于无权图而言，若从源顶点 s 到某个顶点 v 存在路径，则二者之间的最短路径长度 $\delta(s, v)$ 为 s 到 v 路径中边/弧数最少的路径，若源顶点 s 到顶点 v 不存在路径，则最短路径 $\delta(s, v) = \infty$ 。

引理1：对于给定的无权图 $G=(V, E)$ ， $\forall s \in V$ ， $\forall (u, v) \in E$ 或 $\forall \langle u, v \rangle \in E$ ，则 $\delta(s, v) \leq \delta(s, u) + 1$ 。

证明：情况1：源顶点 s 如可达顶点 u ，则源顶点 s 也可达顶点 v 。此种情况下， $\delta(s, v)$ 不可能比 $\delta(s, u)$ 加上 u 到 v 的边更长。情况2：源顶点 s 如不可达顶点 u ，则 $\delta(s, u) = \infty$ ，不等式显然成立。



引理2：对于给定的无权图 $G=(V, E)$ ，若BFS以给定源顶点 $s \in V$ 作为初始出发点，对于 $\forall v \in V$ ，BFS计算出来的 $d[v]$ 满足 $d[v] \geq \delta(s, v)$ 。

证明：通过对BFS入队操作次数进行归纳来证明。考虑将源顶点 s 加入到队列 Q_u 之后的场景，此时 $d[s] = 0 = \delta(s, s) \geq \delta(s, s)$ ，所以归纳假设成立。对于归纳步，假设从结点 u 出发沿邻接表或邻接矩阵能搜索到结点 v ，从算法的赋值操作和引理1可知， $d[v] = d[u] + 1 \geq \delta(s, u) + 1 \geq \delta(s, v)$ 。此外由算法可知， $d[v]$ 执行赋值语句时 v 已经被访问且 v 入队，那么， v 不会再次被访问，因此 $d[v]$ 不会再更新。由此可知归纳假设成立。

引理3: 假定BFS在无权图 $G=(V,E)$ 的运行过程中某时刻, 队列 Qu 依次包含顶点 $\{v_1, v_2, \dots, v_r\}$, v_1 为队头, v_r 为队尾。则 $d[v_r] \leq d[v_1] + 1$; 且对于 $i=1, 2, \dots, r-1$, 有 $d[v_i] \leq d[v_{i+1}] + 1$ 。

证明: 我们仍通过BFS算法中入队操作次数来进行归纳证明。

队列 Qu 初始仅仅包含顶点 s , 定理成立。

对于归纳步, 我们要证明在入队操作和出队操作时, 引理3都成立。如果队头顶点 v_1 出队: 情况1, 队列空, 引理3直接成立; 情况2: 队列不空, v_2 成为新队头, 根据归纳假设, $d[v_1] \leq d[v_2]$, 同时我们还有 $d[v_r] \leq d[v_1] + 1 \leq d[v_2] + 1$, 且余下不等式不受影响, 因此出队操作后引理3成立。

对于入队后的情况, 我们从算法中可知, 查找顶点 w 邻接顶点时, 假设任一新邻接顶点 v 入队列 Qu , 则新顶点 v 入队后即成为队列中为 v_{r+1} , 此时顶点 w 已经出队, 新队头 v_1 满足 $d[v_1] \geq d[w]$ 。因此, $d[v_{r+1}] = d[v] = d[w] + 1 \leq d[v_1] + 1$ 。根据归纳假设, 我们还有 $d[v_r] \leq d[w] + 1$, 因此 $d[v_r] \leq d[w] + 1 = d[v] = d[v_{r+1}]$, 且余下不等式不受影响, 因此入队操作后引理3成立。

引理4: 假定无权图在执行BFS时, 顶点 v_i 和 v_j 都加入到队列 Qu 中, 且 v_i 先入队, 则在 v_j 入队时, 有 $d[v_i] \leq d[v_j]$; 即先入队列者通过BFS计算的路径较短。

证明: 由引理3, 以及BFS过程中最多只更新任意结点 d 值1次, 因此结论成立。

定理5（广度优先遍历的正确性）：对于给定的无权有向图或无向图 $G=(V,E)$ ，假定BFS以给定源顶点 $s \in V$ 作为出发顶点，有如下结论成立：

- ① BFS能发现从源顶点 s 可达的所有顶点 $v \in V$ ；
- ② 在算法中止时，对于所有 s 可达或不可达的顶点 $v \in V$ ，有 $d[v] = \delta(s, v)$ ，即：通过BFS计算的 $d[v]$ 为最短路径；
- ③ 对于任一可达的顶点 $v (v \neq s)$ ，从 s 到 v 的最短路 s 到 v 的直接前驱顶点 w （即由 w 通过查找邻接顶点而BFS访问到 v ）的最短路径+边 (w, v) ，即 $d[v] = d[w] + 1$ 。

证明：使用反证法。假定某个 v 顶点获得的 $d[v]$ 值并不等于其最短路径的距离，其最短路径为 $\delta(s, v)$ ，显然 $v \neq s$ 。根据引理2，有 $d[v] \geq \delta(s, v)$ ，因此 $d[v] > \delta(s, v)$ 。另外，结点 v 必是从 s 可达的，否则将有 $\delta(s, v) = \infty \geq d[v]$ 。设 w 是 s 到 v 的直接前驱顶点， $\delta(s, v) = \delta(s, w) + 1$ 。因为 $\delta(s, w) < \delta(s, v)$ ，且算法对顶点 v 的访问方式，可知 $d[w] = \delta(s, w)$ 。合并上述结果，有： $d[v] > \delta(s, v) = \delta(s, w) + 1 = d[w] + 1 \dots \textcircled{1}$ 。

考虑BFS算法将 v 从队列 Qu 中出队之时，根据算法实现过程， v 已经被访问，依照算法的赋值语句，有 $d[v] = d[w] + 1$ 。这与 $\textcircled{1}$ 矛盾。

因此，我们有所有 $v \in V$ ， $d[v] = \delta(s, v)$ 。所有 s 可达顶点 v 都必定被访问到，否则 $\infty = d[v] > \delta(s, v)$ 。同时注意到： v 的直接前驱顶点为 w ， $d[v] = d[w] + 1$ ；因此，从 s 到 v 的其中一条最短路径为 s 到 v 的直接前驱顶点 w 的最短路径+边 (w, v) 。

[广度优先生成树性质]

BFS算法对图进行搜索过程中将创建一颗广度优先生成树，该树对应着BFS算法完成时p数组保存的值。

定义1: 对于图G和源顶点s, **广度优先搜索的前驱子图**是 $G_p=(V_p, E_p)$, 其中 $V_p=\{v \in V, p[v] \neq -1\} \cup \{s\}$, $E_p=\{(p[v], v): v \in V_p - \{s\}\}$.

性质: 若 V_p 由所有从源顶点s出发可达的顶点构成, 且对于所有 $v \in V_p$, 子图 G_p 包含一条从源顶点s到顶点v的唯一简单路径, 则该路径也是从源顶点s到顶点v的一条最短路径, 而且前驱子图 G_p 也是一棵**广度优先生成树**。

定理: 对于图 $G=(V, E)$, 利用BFS算法计算的p数组构造的前驱子图是一棵广度优先生成树。

证明: 算法在赋值 $p[v]=u$ 时, 当且仅当 $(u, v) \in E$, 并且 $\delta(s, v) < \infty$; 即若源顶点s到顶点v可达, 且 V_p 由所有从源顶点s出发可达的顶点构成, 由于 G_p 是一棵树, 因此该树从源顶点s到每个可达顶点v必定存在唯一一条简单路径。通过递归地应用广度优先遍历的正确性定理, 我们可知每条简单路径也必定是一条最短路径。

3 图的遍历小结

- ❖ 深度优先遍历算法借助于栈结构实现；广度优先遍历算法借助于队列结构实现。
- ❖ 图的遍历序列与算法和存储方式有关。
- ❖ 对于非连通图，通过遍历可求得各连通分量。
- ❖ 广度优先遍历，可求无权图的最短路径。

[示例] 假设图采用邻接表存储，写出基于DFS和BFS遍历的算法来判定顶点*i*和*j*之间有没有路径存在。

算法思想：先置全局变量visit[MAXV]=FALSE;从顶点*i*开始做深度优先遍历或广度优先遍历，遍历结束时若visit[j]==TRUE;则说明顶点*i*与*j*之间有路径，否则无路径。

深度优先

```
bool visit[MAXV];
Bool DFSTavel(Agraph *G, int i, int j)
{
    for(int k =0; k<MAXV; k++)
        visit[k] = FALSE;
    DFS(G,i);
    return (visit[j]==FALSE)?FALSE:TRUE;
}
```

广度优先

```
bool visit[MAXV];
Bool BFSTavel(Agraph *G, int i, int j)
{
    for(int k =0; k<MAXV; k++)
        visit[k] = FALSE;
    BFS(G,i);
    return (visit[j]==FALSE)?FALSE:TRUE;
}
```

7.4 图的连通性

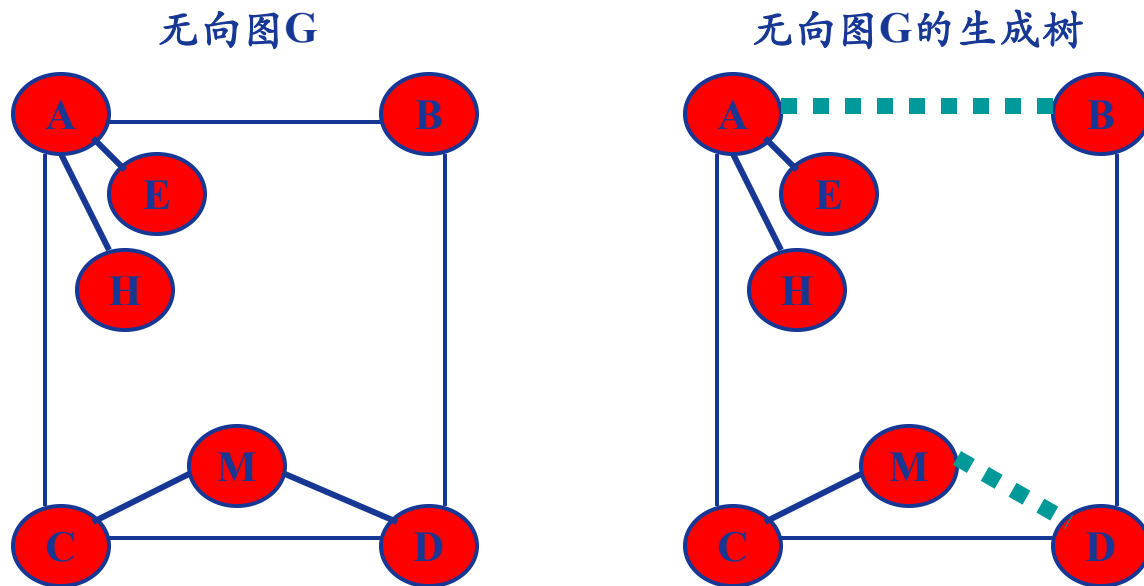
1 无向图的连通分量和生成树

[连通图的生成树]

- ❖ **生成树**是图G的一个子图，该子图是包含G的所有顶点的树(或者是自由树—未确定根结点)。
- ❖ 若图G有n个顶点，则其生成树必具有n-1条边。
- ❖ 生成树是包含图中所有顶点的极小连通子图。
- ❖ 生成树可以通过对图的深度/广度优先遍历而得到，称之为深度/广度优先生成树。

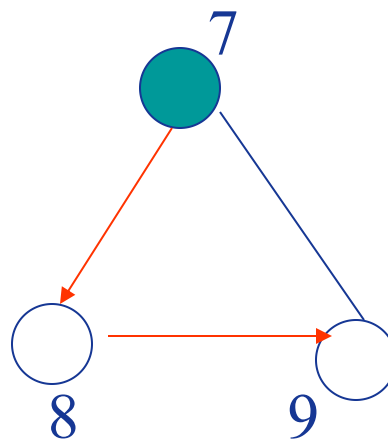
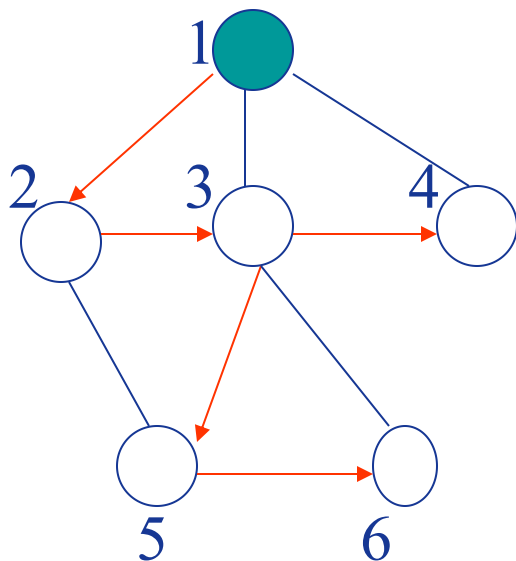
在算法中，访问一个结点时 v_j ，同时记录它的父结点 v_i ， (v_i, v_j) 是生成树的一条边，即树边。

- ❖ 一般情况，BFS(广度优先搜索)生成树的树高小于DFS(深度优先搜索)生成树的高度。
- ❖ 一个图的生成树是不唯一的
 - 从不同的顶点出发
 - 采用不同的存储结构和存储顺序



[非连通图的生成森林]

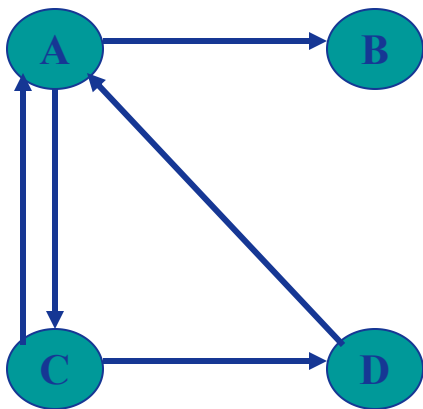
- ❖ 一个连通分量及其遍历时走过的边构成一棵生成树。
- ❖ 非连通图的各连通分量的生成树组成**生成森林**。



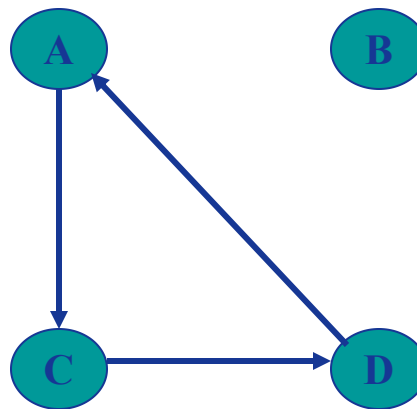
2 有向图的强连通分量的求法

- 强连通图：有向图图 G 的任意两点之间都是连通的，则称 G 是强连通图。
- 强连通分量：极大连通子图

有向图G



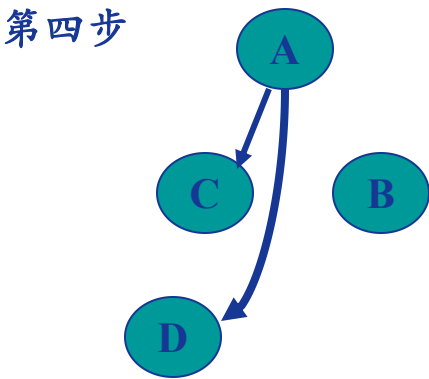
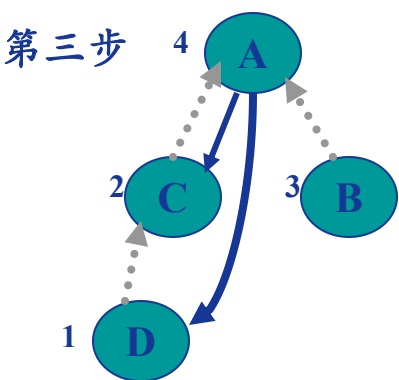
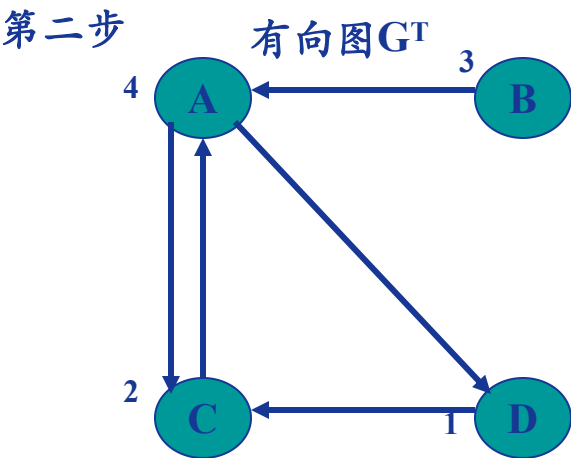
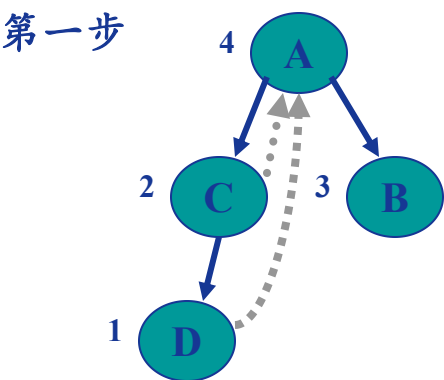
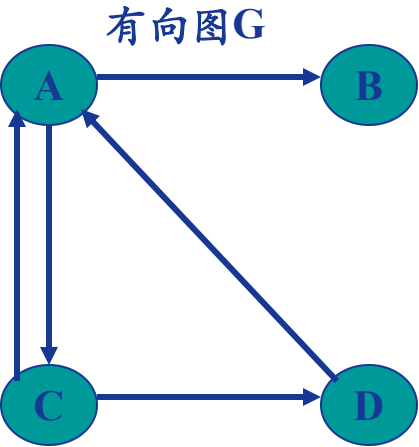
有向图G的两个强连通分量



[求强连通分量的算法]

- ①对有向图 $G=(V,E)$ 进行深度优先搜索，按回溯次序给结点进行编号。最先出现回溯的结点的编号为 1，其它结点的编号按回溯次序逐次增大 1。
- ②将有向图 G 的所有的有向边进行倒置，构造新的有向图记为 G^T ， $G^T=(V, E^T)$ ， $E^T=\{<u,v>: <v,u>\in E\}$ 。
- ③根据步骤①对结点进行的编号，选取最大编号的结点。从该结点出发，在有向图 G^T 上进行深度优先遍历。如果没有访问到所有的结点，则从剩余的那些结点中选取编号最大的结点再进行深度优先遍历。直至所有的结点都被访问到为止。
- ④在最后得到的生成森林中的每一棵生成树，都是有向图 G 的强连通分量顶点集。

•实例



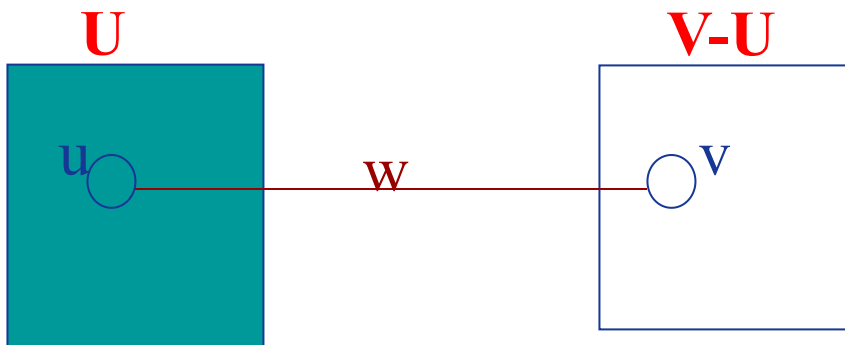
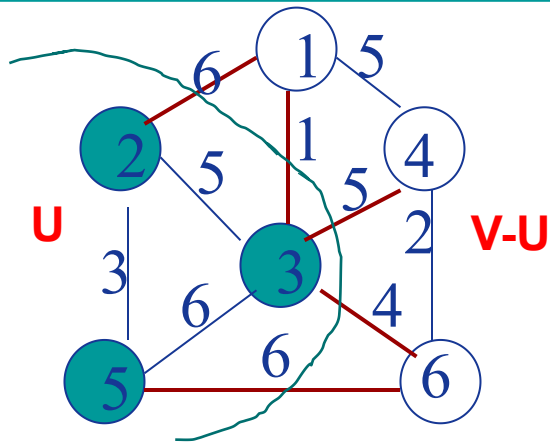
二棵生成树的结点是有向图G
的两个强连通分量的顶点集：
{A、C、D}及{B}

2. 最小生成树

[概述]

- ❖ 由一个连通带权无向图 $G=(V, E)$ ，对每条边 $(u,v) \in E$ 赋予权值 $w(u,v)$ ，找到一个无环子集 $T \subseteq E$ ，能将所有顶点连接起来，且 T 各边的权值之和最小，即 $w(T) = \sum_{(u,v) \in T} w(u,v)$ 的值最小。由于 T 无环且连通，因此 T 必为树，称这样的树为**最小生成树**，记为MST。
- ❖ **应用场景**：在电子电路设计中，常常需要将不同组件的针脚连在一起，要连接 n 个针脚，可以使用 $n-1$ 根连线，我们希望连线长度最短。这类布线问题可以看成是一个连通无向图 $G=(V, E)$ ， V 是针脚集合， E 是针脚之间可能的连线。
- ❖ 两种最小生成树算法都是基于贪心算法的，只是贪心策略不一样。贪心是在多个可能中选择一个，一般不能保证得到一个全局最优解。但对于最小生成树问题，可证明这两种贪心策略是能找到最优解的。

MST性质（贪心）： 设 $G=(V, E)$ 是一个连通的带权图，权重函数为 w ， U 是 V 的真子集，若边 $(u, v)[u \in U, v \in V-U]$ 是 E 中所有一个端点在 U 内，一个端点不在 U 内的边中权值最小的一条边(轻边/安全边)，则一定存在 G 的一棵最小生成树包括此边。



证明：假定存在一棵不包括轻边 (u, v) 在内的最小生成树，设其为 T 。

①现在来构造另外一棵最小生成树 T' ，将边 (u, v) 添加到树 T ，则 (u, v) 和 u 到 v 的简单路径 p 将构成一个回路。因此，必定存在另一条边 (u', v') ，且 $u' \in U, v' \in V-U$ 。删去边 (u', v') ，得到另一棵生成树 $T' = T - \{(u', v')\} \cup \{(u, v)\}$ ，新的生成树 T' 是树的理由：**连通且无回路。**

②接下来证明 T' 是一棵最小生成树，由于 $w(u, v) \leq w(u', v')$ ；因此，有 $w(T') = w(T) + w(u, v) - w(u', v') \leq w(T)$ 。因此， T' 是一棵最小生成树。和原假设矛盾。

[Prim算法]

利用prim算法从指定顶点 v 开始构造最小生成树的步骤如下：

(1)初始化 $U=\{v\}$ ，将 v 到其他顶点的所有边作为候选边。

(2)重复以下2个步骤 $n-1$ 次，直到其他 $n-1$ 个顶点加入到 U 中。

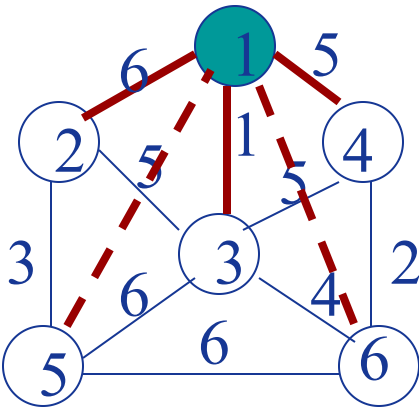
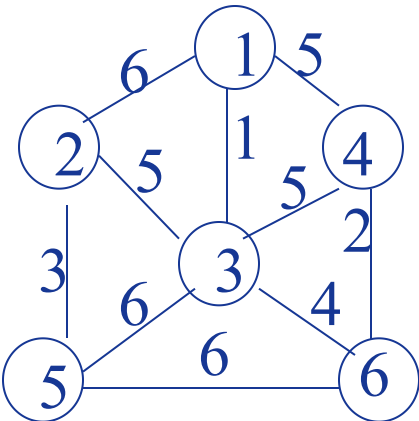
①从候选边中挑选权值最小的边作为树边输出，设该边在 $V-U$ 中的顶点为 k ，将 k 加入到集合 U 中。

②考察当前 $V-U$ 集合中的所有顶点 j ，修正侯选边：若 $w(k, j)$ 小于原来的侯选边权值，则 (k, j) 取代后者作为新的侯选边。

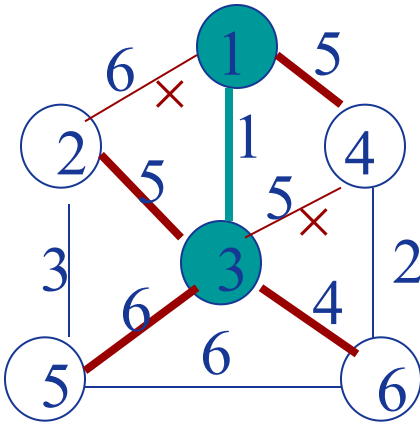
说明： prim算法是将顶点分为两个部分， U 和 $V-U$ 集合，利用MST性质，执行贪心策略：每次从中选择一个连通 U 和 $V-U$ 集合中权值最小的边作为树边加入，再调整 U 和 $V-U$ 集合，和连通两个顶点集的侯选边。

[示例]

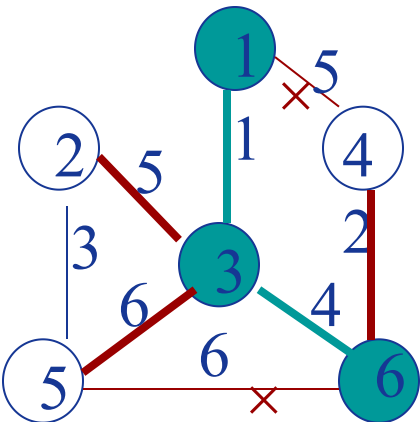
n=6



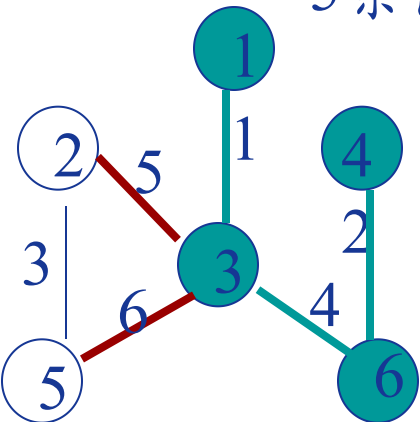
5条候选边



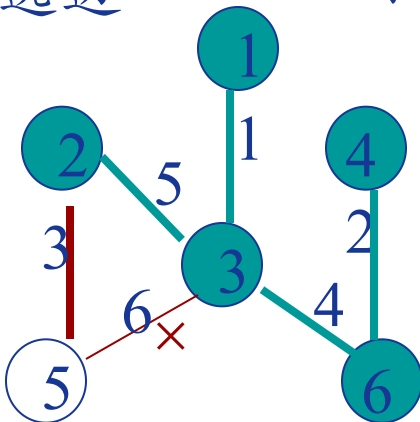
4条候选边



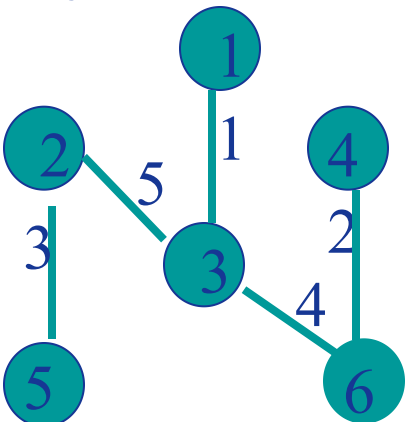
3条候选边



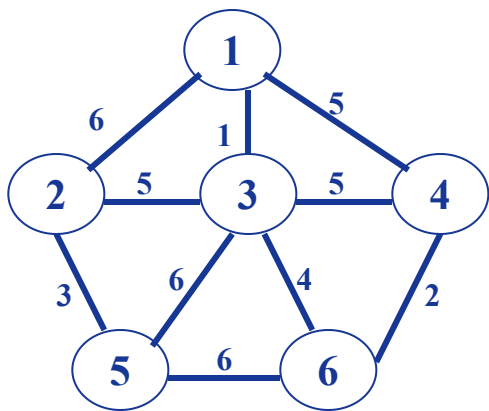
2条候选边



1条候选边



[Prim算法的数据结构]



图G

主存储结构:

带权邻接矩阵或邻接表记录图

辅助变量的定义

```
typedef struct
```

```
{
```

```
    int lowcost;
```

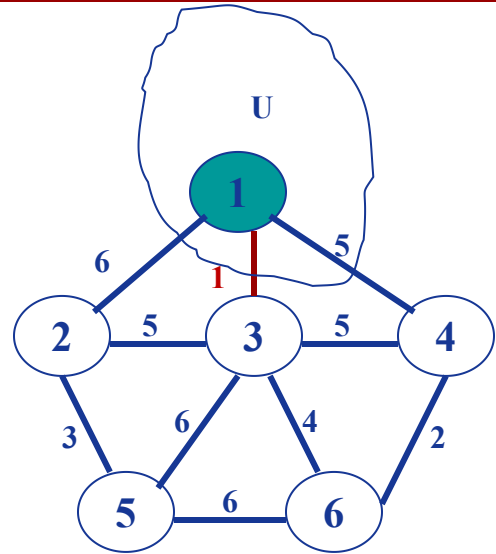
```
    int adjvex;
```

```
} EdgeInfo;
```

```
EdgeInfo closedge[MAXV];
```

注: $\text{closedge}[k].\text{lowcost} = 0$

表示顶点k已在U集合中。

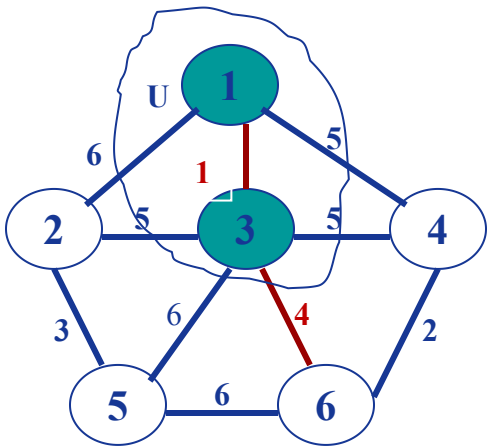


图G

数组: closedge[6]

1	0	
2	6	1
3	1	1
4	5	1
5	∞	
6	∞	

lowcost adjvex

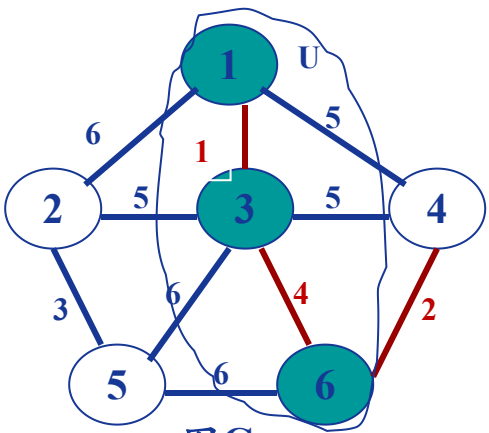


图G

数组: closedge[6]

1	0	
2	5	3
3	0	
4	5	1
5	6	3
6	4	3

lowcost adjvex

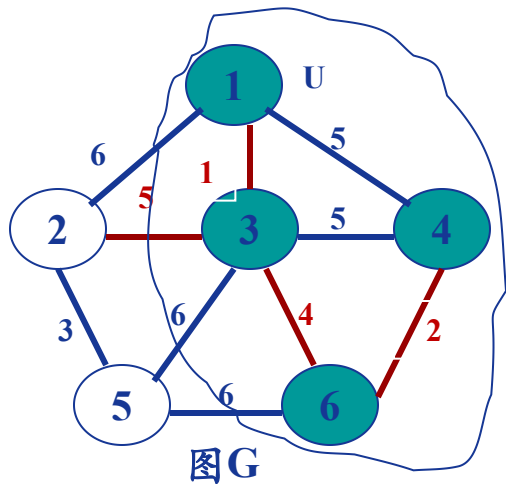


图G

数组: closedge[6]

1	0	
2	5	3
3	0	
4	2	6
5	6	3
6	0	

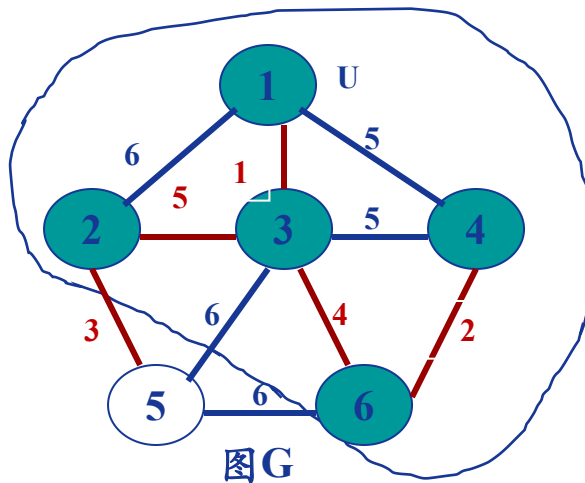
lowcost adjvex



数组: closedge[6]

1	0	
2	5	3
3	0	
4	0	0
5	6	3
6	0	

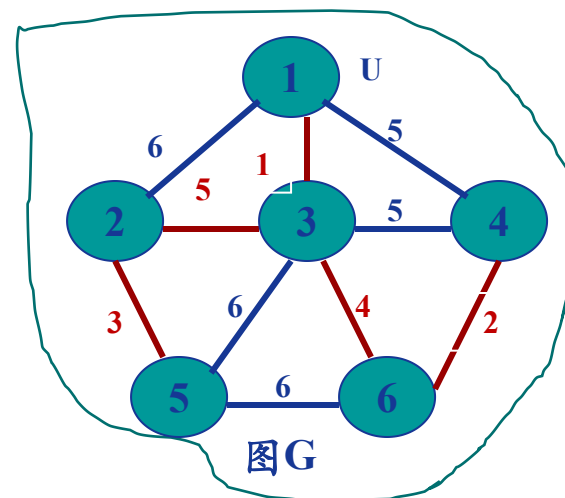
lowest adjvex



数组: closedge[6]

1	0	
2	0	
3	0	
4	0	0
5	3	2
6	0	

lowest adjvex



数组: closedge[6]

1	0	
2	0	
3	0	
4	0	
5	0	
6	0	

lowest adjvex

【算法步骤】

- 1) 初始化 $\text{closedge}[v](v=1..n)$ $O(n)$
 - 2) 重复 $n-1$ 次以下操作：
 - 2.1) 在 $\text{closedge}[v](v=1..n)$ 中选择最小且非0的 lowcost ，记录其 v 值(设为 k)和相应的 vex ; $O(n^2)$
 - 2.2) 输出该边 (vex, k) ; $O(n)$
 - 2.3) 顶点 k 并入 U 集: $\text{closedge}[k].\text{lowcost}=0$; $O(n)$
 - 2.4) 调整候选边集 $\text{closedge}[v](v=1..n)$: $O(n^2)$
 若 $\text{gn}[k, v] < \text{closedge}[v].\text{lowcost}$,
 则更改 $\text{closedge}[v]$: $\text{vex}=k, \text{lowcost}=\text{gn}[k, v]$
-
- $O(n^2)$

【算法特点】 适用于稠密图

思考：若采用邻接表存储，Prime 算法时间复杂度为多少？

[Prim算法的实现-邻接矩阵]

```

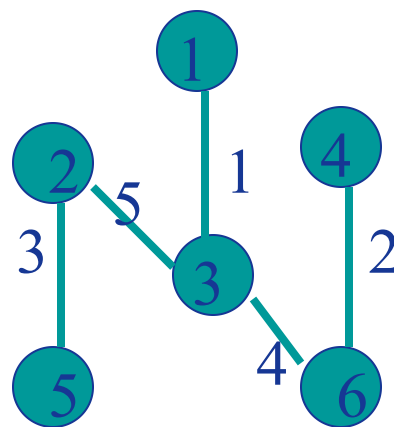
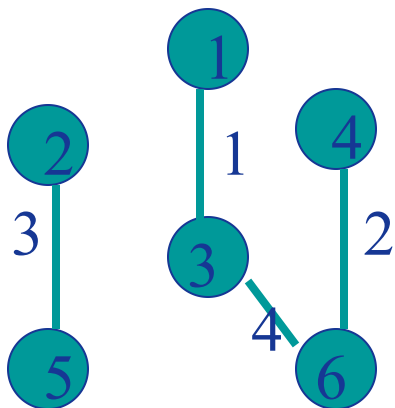
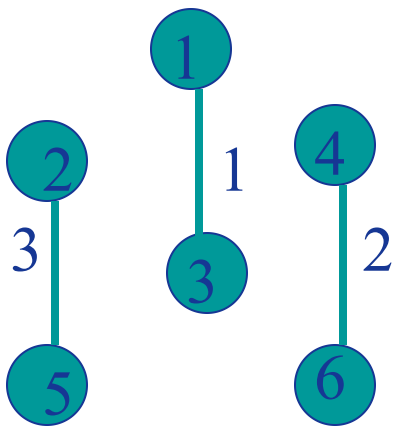
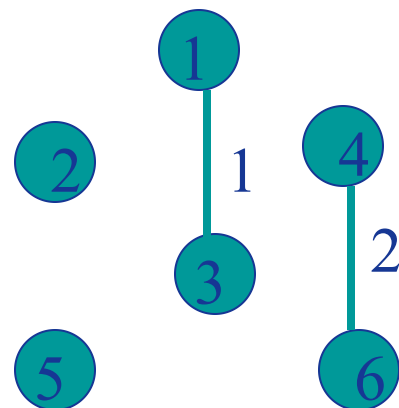
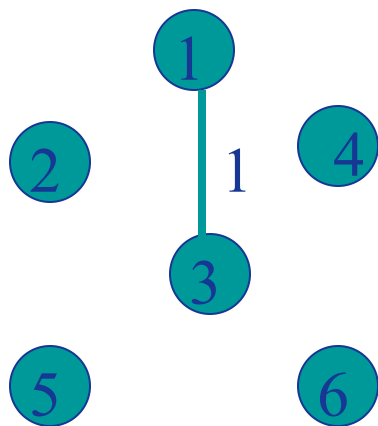
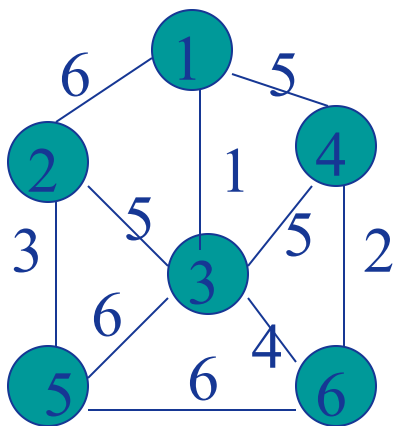
Void Prim(Mgraph g, int v)
{
    EdgeInfo closedge[1,...,MAXV];
    int i, j, k, min;
    for (i=1; i<=g.n; i++) {
        closedge[i].lowcost = g.edges[v][i];
        closedge[i].adjvex = v;
    }
    for (i=2; i<=g.n; i++) { //找出n-1个顶点;
        min = INF; //INF表示理论上的最大值, 无穷
        for (j=1; j<=g.n; j++) //在V-U中寻找离U最近的顶点k
            if (closedge[j].lowcost != 0 & closedge[j].lowcost < min) {
                min = closedge[j].lowcost;
                k = j;
            } // end if
        printf("边(%d,%d)权为%d\n", closedge[k].adjvex, k, closedge[k].lowcost);
        closedge[k].lowcost = 0; //标记顶点k加入U集合
        for (j=1; j <= g.n; j++)
            if (g.edges[k][j] != 0 && g.edges[k][j] < closedge[j].lowcost) { //仅仅考虑V-U集合中的顶点
                closedge[j].lowcost = g.edges[k][j]; //修改closedge数组
                closedge[j].adjvex = k;
            } // end if
    } // end for (i=2; i<=g.n; i++)
}

```

[Kruskal算法]

[示例] n=6

Kruskal算法找轻边/安全边的**贪心策略**是:在所有能连接森林中两棵不同树的边中选择权重最小的边(u,v)



[算法步骤]

- 1) 初始化T: 顶点集=所有顶点, 每个独立的顶点作为一棵树, 边集= $\emptyset$; $O(n)$
- 2) 依权值递增序对图G的边排序, 结果为 $E[1..e]$ $O(\text{elge})$
- 3) 依次检测E中的各边(u,v): $O(\text{elge})$
 - 3.1) 若u和v分属于T中两棵不同的树, 则将该边加入T, 并合并u和v分属的两棵树
 - 3.2) 若T中所有顶点尚未属于一棵树, 转3)

[算法特点] 适用于稀疏图

$O(\text{elge})$

kruskal算法中要解决的关键问题:

- ① 如何为所有边排序: 小根堆
- ② 如何把两棵树合并成一棵: 并查集

[利用小根堆和并查集来实现kruskal算法]

①首先，利用小根堆来存放E中的所有的边，堆中每个结点的格式为

<i>tail</i>	<i>head</i>	<i>cost</i>
边的两个顶点位置		边的权值

②在构造最小生成树的过程中，小根堆中存放剩余的边，并且利用并查集的运算检查依附于一条边的两个顶点 *tail*、*head* 是否在同一个连通分量(即并查集的同一个子集合)上，是则舍去这条边；否则将此边加入 *T*，同时将这两个顶点放在同一个连通分量上。随着各边逐步加入到最小生成树的边集合中，各连通分量也在逐步合并，直到形成一个连通分量为止。

	0	1	2	3	4	5	6	
0	0	28	∞	∞	∞	10	∞	0
28	0	16	∞	∞	∞	∞	14	1
∞	16	0	12	∞	∞	∞	∞	2
∞	∞	12	0	22	∞	18	∞	3
∞	∞	∞	22	0	25	24	∞	4
10	∞	∞	∞	25	0	∞	∞	5
∞	14	∞	18	24	∞	0	0	6

(a) 邻接矩阵表示

H	0	1	2	3	4	5	6	7	8
建堆	0, 1, 28	0, 5, 10, 0, 1, 28	0, 5, 10, 0, 1, 28, 1, 2, 16	0, 5, 10, 1, 6, 14, 1, 2, 16, 0, 1, 28	0, 5, 10, 2, 3, 12, 1, 2, 16, 0, 1, 28, 1, 6, 14	0, 5, 10, 2, 3, 12, 1, 2, 16, 0, 1, 28, 1, 6, 14, 3, 4, 22	0, 5, 10, 2, 3, 12, 1, 2, 16, 0, 1, 28, 1, 6, 14, 3, 4, 22, 3, 6, 18	0, 5, 10, 2, 3, 12, 1, 2, 16, 4, 5, 25, 1, 6, 14, 3, 4, 22, 3, 6, 18, 0, 1, 28	0, 5, 10, 2, 3, 12, 1, 2, 16, 4, 6, 24, 1, 6, 14, 3, 4, 22, 3, 6, 18, 0, 1, 28, 4, 5, 25

取边	0, 5, 10	2, 3, 12	1, 2, 16	4, 6, 24	1, 6, 14	3, 4, 22	3, 6, 18	0, 1, 28	4, 5, 25
初态	0, 5, 10	2, 3, 12	1, 6, 14	1, 2, 16	4, 6, 24	4, 5, 25	3, 4, 22	3, 6, 18	0, 1, 28
加入边 (0, 5, 10)	1, 6, 14	4, 6, 24	1, 2, 16	0, 1, 28	4, 5, 25	3, 4, 22	3, 6, 18	0, 1, 28	4, 5, 25
加入边 (2, 3, 12)	1, 2, 16	4, 6, 24	3, 6, 18	0, 1, 28	4, 5, 25	3, 4, 22	0, 1, 28	4, 5, 25	3, 4, 22
加入边 (1, 6, 14)	3, 6, 18	4, 6, 24	3, 4, 22	0, 1, 28	4, 5, 25	0, 1, 28	4, 5, 25	0, 1, 28	4, 5, 25
加入边 (1, 2, 16)	3, 4, 22	4, 6, 24	4, 5, 25	0, 1, 28	0, 1, 28	0, 1, 28	0, 1, 28	0, 1, 28	0, 1, 28
(3, 6, 18) × 加入边 (3, 4, 22)	4, 6, 24	0, 1, 28	4, 5, 25	0, 1, 28	0, 1, 28	0, 1, 28	0, 1, 28	0, 1, 28	0, 1, 28
(4, 6, 24) × 加入边 (4, 5, 25)	4, 5, 25	0, 1, 28	0, 1, 28	0, 1, 28	0, 1, 28	0, 1, 28	0, 1, 28	0, 1, 28	0, 1, 28

(b) 最小堆

F	0	1	2	3	4	5	6
初态	-1	-1	-1	-1	-1	-1	-1
加入边 (0, 5, 10)	-2	-1	-1	-1	-1	0	-1
加入边 (2, 3, 12)	-2	-1	-2	2	-1	0	-1
加入边 (1, 6, 14)	-2	-2	-2	2	-1	0	1
加入边 (1, 2, 16)	-2	-4	1	2	-1	0	1
(3, 6, 18) × 加入边 (3, 4, 22)	-2	-5	1	2	1	0	1
(4, 6, 24) × 加入边 (4, 5, 25)	1	-7	1	2	1	0	1

(c) 并查集

[kruskal参考算法]

```

void Graph<string, float>::
Kruskal ( MinSpanTree &T ) {
    MSTEdgeNode e;           //边结点辅助单元
    MinHeap<MstEdgeNode> H(CurrentEdges);
    int NumVertices = VerticesList.last;    //顶点个数
    UFSets F (NumVertices);    //并查集F 并初始化
    for ( int u = 0; u < NumVertices; u++ ) //邻接矩阵
        for ( int v = i+1; v < NumVertices; v++ )
            if ( Edge[u][v] != MAXINT ) { //检出所有边
                e.tail = u; e.head = v;
                e.cost = w; H.Insert (e);    //插入最小堆
            }
    int count = 1;           //最小生成树加入边数的计数
    while ( count < NumVertices ) { //
        e = H.RemoveMin ();    //从堆中退出一条边
        u = F.Find ( e.tail );    //检测两端点的根
        v = F.Find ( e.head );
        if ( u != v ) { //根不同, 不在同一连通分量上
            F.Union ( u, v );    //合并
            T.Insert ( e );    //该边存入最小生成树 T 中
            count++;
        }
    }
}

```

[kruskal算法时间性能分析]

- ①在建立最小堆时需要检测邻接矩阵，计算的时间代价为 $O(n^2)$ 。且做了 e 次堆插入操作，每次插入调用了一个堆调整算法 $SiftUp()$ 算法，因此堆插入需要的时间代价为 $O(e \log_2 e)$ 。
- ②在构造最小生成树时，最多调用了 e 次出堆操作 $RemoveMin()$ ， $2e$ 次并查集的 $Find()$ 操作， $n-1$ 次 $Union()$ 操作，计算时间分别为 $O(e \log_2 e)$ ， $O(e \log_2 n)$ 和 $O(n)$ 。
- ③对于无向连通图而言 $e \geq n-1$ ，因此总的计算时间为 $O(e \log_2 e + e \log_2 n + n) = O(e \log_2 e)$

[Kruskal算法的修改]

用“破圈法”求解带权连通无向图的一棵最小代价生成树。所谓“破圈法”就是“任取一圈（环），去掉圈上权最大的边”，反复执行这一步骤，直到没有圈为止。**算法重点在于如何将边排序以及如何判定环。**

[算法描述]

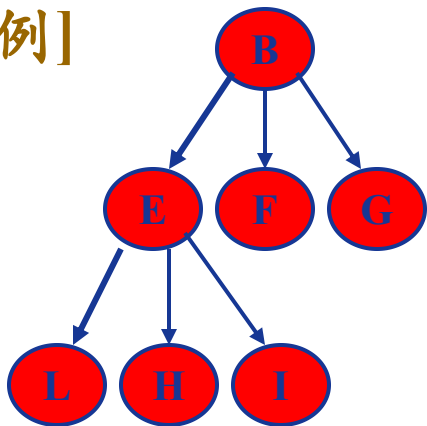
- ①将图中所有边按权重从大到小排序为 $(e_1, e_2, \dots, e_m)$
- ② $i=1$
- ③ while ($i < \text{顶点数}$)
 - {
 - 从图中删去 e_i ;
 - 若图不再连通，则恢复 e_i ;
 - $i=i+1$;
 - }

7.5 有向无环图的应用

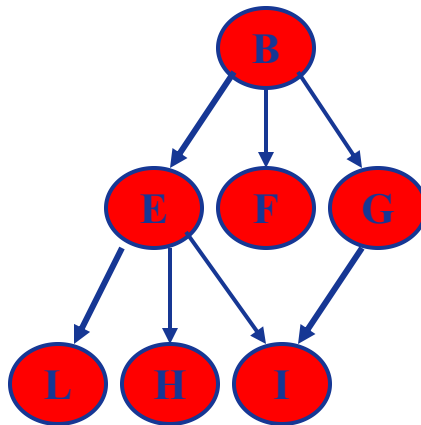
BUPT

1、何为有向无环图 (DAG 图)

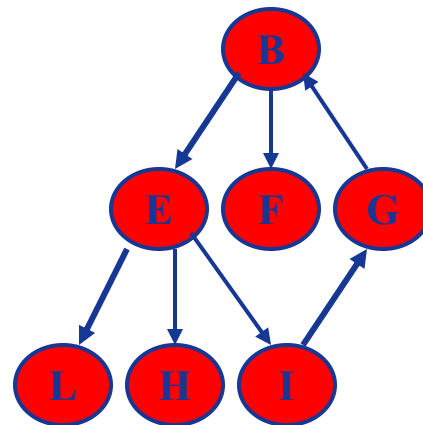
[实例]



有向树



DAG图



有向图 (含环)

[用途]: 描述工程项目或系统进行的工具

AOV 网络: 定义顶点为活动, 有向边的指向表示活动执行的次序。



AOE网络: 定义顶点为事件, 单位是时刻。有向边定义为活动, 它的权值定义为活动进行所需要的时间。



❖ 思考：有向无环图以邻接矩阵存储，按照何种规律对顶点进行编号就能使其邻接矩阵成为下三角矩阵？

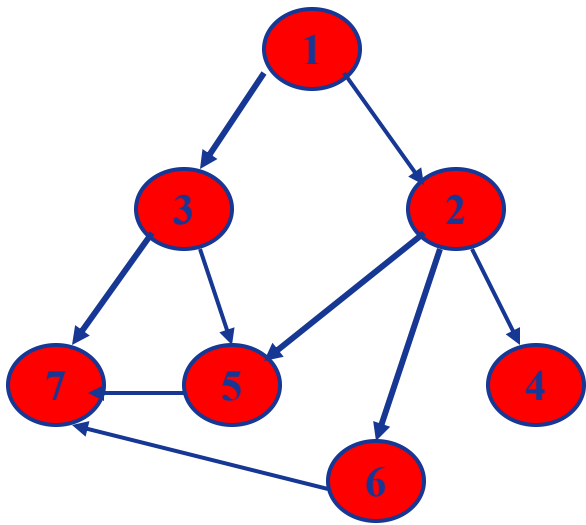
编号规则：让弧尾顶点的编号大于弧头顶点的编号。

2 拓扑排序

[问题目标]

当一个任务可以划分成若干个子任务/子活动/子事件，其中的任何子任务可能又以另外的一些子任务作为先决条件时，如何排定子任务的执行顺序，达到整体任务的完成。

[示例]



课程之间的依赖关系
其中，1代表数学，2代表程序设计，3代表离散数学，4代表汇编程序设计，5代表数据结构，6代表结构程序设计，7代表编译原理。

拓扑序列：对有向无环图G，其拓扑序列是G中所有顶点的一种线性次序。该次序满足：若G中包含弧 $\langle u,v \rangle$ ，则结点u在拓扑序列中必定位于顶点v的前面。



AOV网(顶点活动网络)—顶点表示活动，弧表示活动间的优先关系

2.1 无前趋的顶点优先算法

[算法原理]

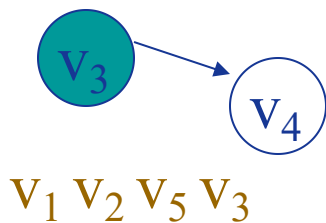
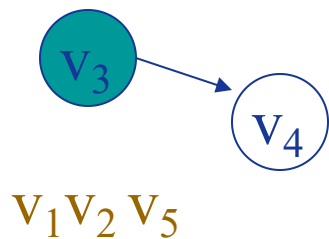
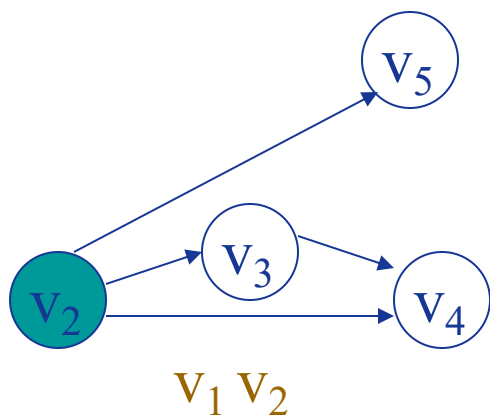
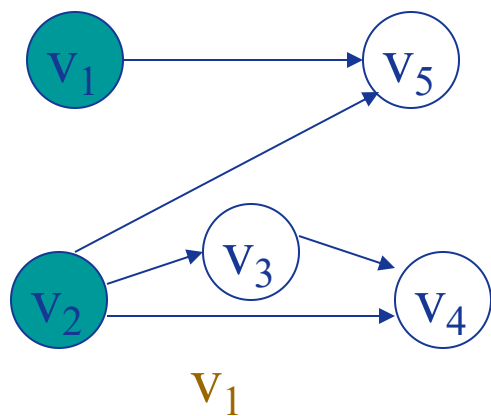
在一个拓扑序列中，每个顶点必定出现在它的所有前趋顶点之后。

[算法思想]

1. 选择一个入度为0的顶点(无前趋的顶点)，输出它
 2. 删去该顶点及其关联的所有出边
- 重复上述两步，直至图中不再有入度为0的顶点为止。
若所有顶点均被输出，则排序成功，
否则图中存在有向环。

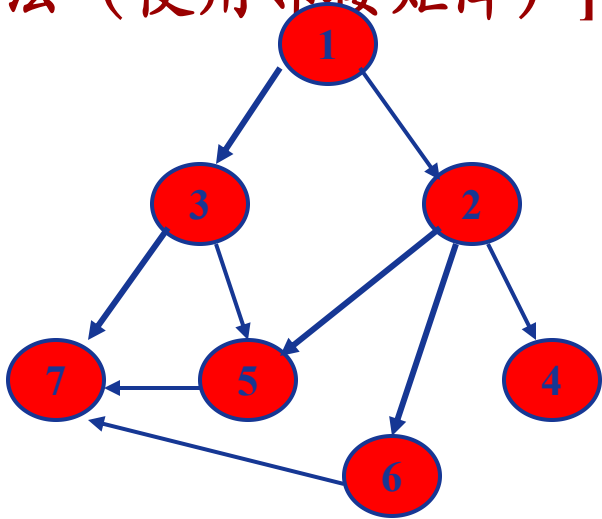
若该有向图不是无环图，则不存在拓扑排序。可以用来判断一个有向图是否存在回路。

[例]



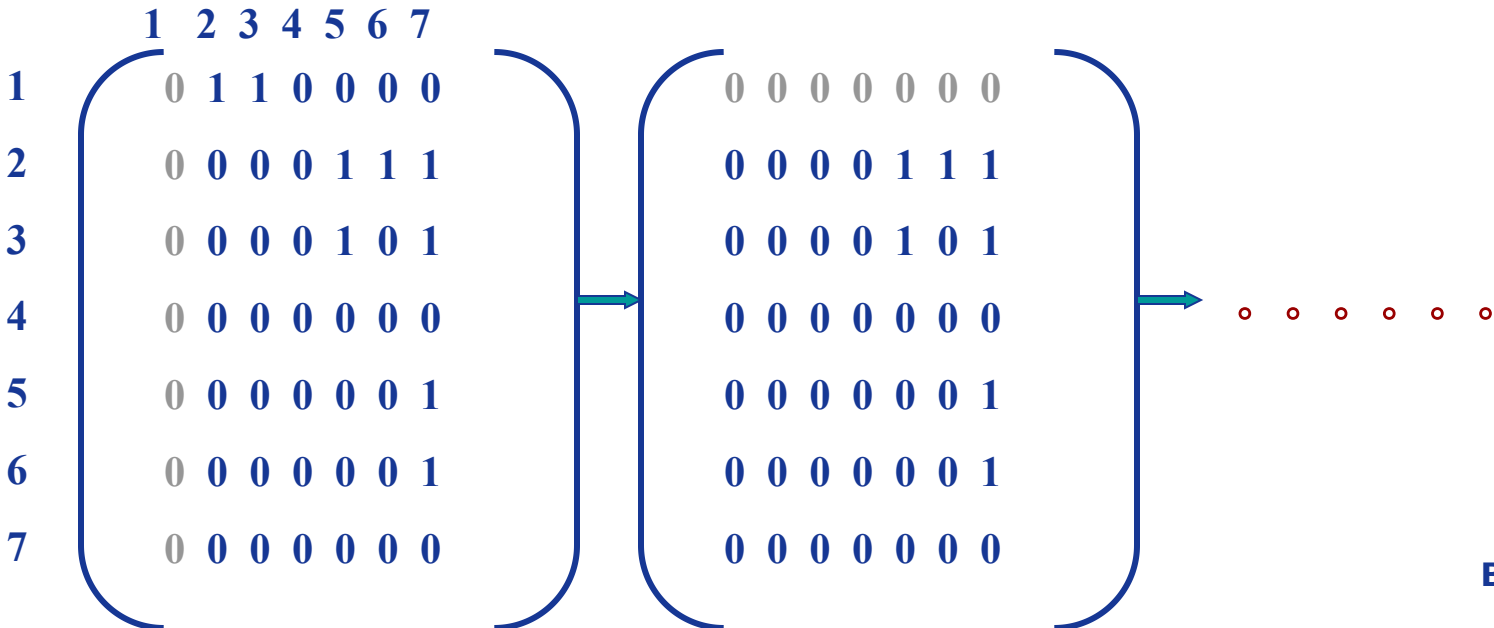
有向无环图的拓扑序列不唯一

[算法（使用邻接矩阵）]

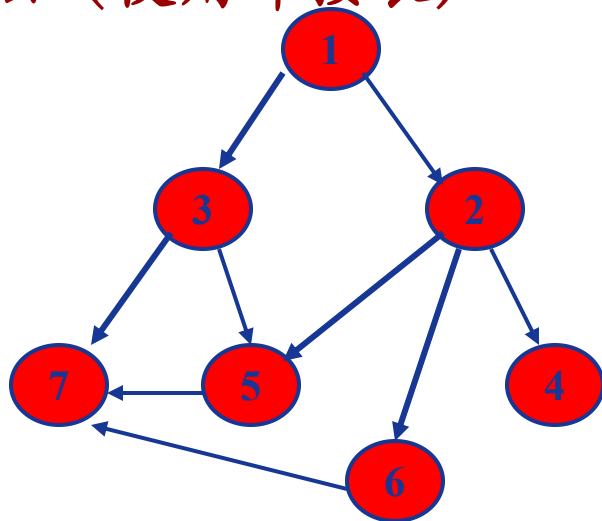


算法的执行步骤：

- 1、找到全为零的第j 列，输出 j
- 2、将第j 行的全部元素置为零
- 3、反复执行 1和2步到所有元素输出完毕。



算法（使用邻接表）



算法的执行步骤：

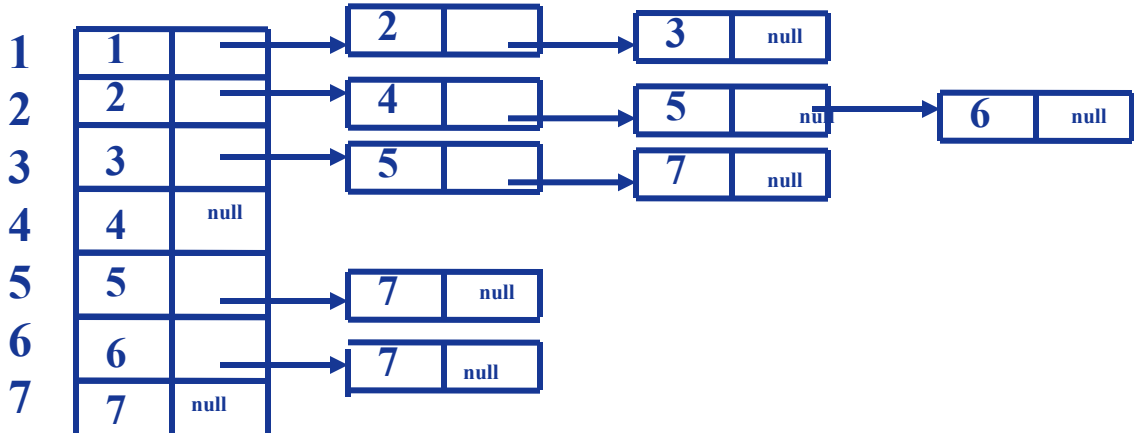
- 1、用一个数组记录每个结点的入度。将入度为零的结点进栈。
- 2、将栈中入度为零的结点V输出。
- 3、根据邻接表找到结点V的所有的邻接结点，并将这些邻接结点的入度减一。如果某一结点的入度变为零，则进栈。
- 4、反复执行 2、3；直至栈空为止。

.....

执行结束，如果输出结点数等于图的结点总数，则有向图有环，否则有向图有环。

栈（队列） indegree

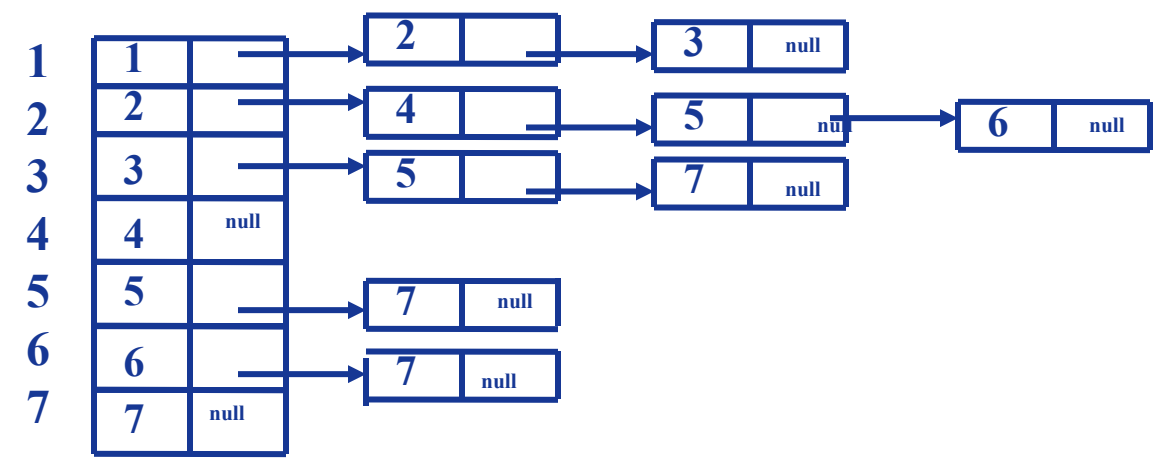
	1	0
	2	1
	3	1
	4	1
	5	2
	6	1
1	7	3



//邻接表存储

bool Topologicalsort(Agraph *G) //有拓扑序列，打印拓扑序列，返回TRUE,否则返回FALSE.

```
{
    findinDegree(G, indegree); //计算每个结点的入度，用indegree数组返回。
    Initstack(S); //栈S初始化操作
    for(i = 1; i <= G.n; i++)
        if (indegree[ i ]==0) Push(S, i); //顶点i入度为0，入栈
    count = 0;
    while (!StackEmpty(S)) //栈不空
    {
        Pop(S, i); //栈顶元素出栈，用i保存栈顶元素；
        Printf(i, adjlist[i]. Data); //打印顶点i的值；若无环，打印的就是拓扑序列
        ++count; //拓扑序列中元素个数加1；
        for (p=G->adjlist[i].firstarc; p; p=p->nextarc)//寻找顶点i的邻接顶点
        {
            k= p->adjvex; //顶点i的邻接顶点k;
            if ((- indegree [ k ]) ==0) Push(S, k); //顶点k入度-1之后若==0，则k入栈；
        }
    }
    if (count < G.n) return FALSE; //栈空，但输出顶点数不等于G.n，有环；
    else return TRUE;
}
```



(1)

	Indegree	栈
1	0	
2	1	
3	1	
4	1	
5	2	1
6	1	
7	3	

(2)

	Indegree	栈
1	0	
2	0	
3	0	
4	1	3
5	2	2
6	1	
7	3	1

(3)

	Indegree	栈
1	0	
2	0	
3	0	
4	1	
5	1	2
6	1	
7	2	

1,3

(4)

	Indegree	栈
1	0	
2	0	
3	0	6
4	0	5
5	0	4
6	0	
7	2	

1,3,2

(5)

	Indegree	栈
1	0	
2	0	
3	0	
4	0	
5	0	5
6	0	4
7	1	

1,3,2,6

(6)

	Indegree	栈
1	0	
2	0	
3	0	
4	0	7
5	0	4
6	0	
7	0	

1,3,2,6,5

(7)

	Indegree	栈
1	0	
2	0	
3	0	
4	0	
5	0	4
6	0	
7	0	

1,3,2,6,5,7,4

(8)

	Indegree	栈
1	0	
2	0	
3	0	
4	0	
5	0	
6	0	
7	0	

1,3,2,6,5,7,4,3

1,3,2,6,5,7

2.2 无后继的顶点优先

[算法原理]

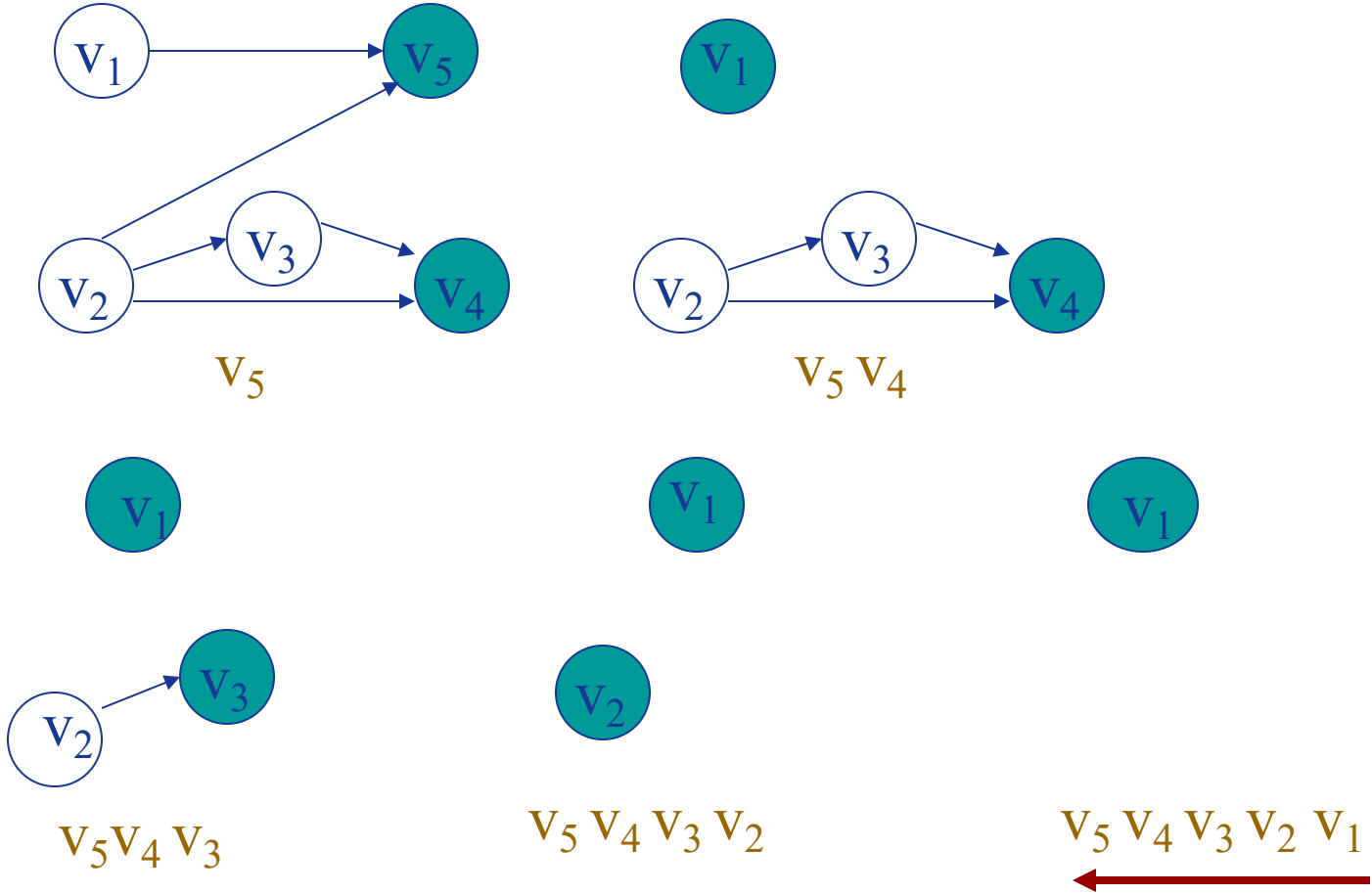
在一个拓扑序列中，每个顶点必定出现在它的所有后继顶点之前。

[方法一 算法思想]

(按逆拓扑次序生成顶点序列)

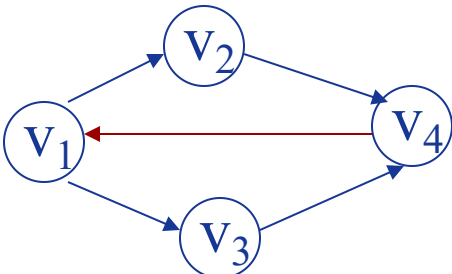
1. 选择一个出度为0的顶点(无后继的顶点)，输出它
 2. 删去该顶点及其关联的所有入边
- 重复上述两步，直至图中不再有出度为0的顶点为止。
若所有顶点均被输出，则排序成功，
否则图中存在有向环。

[例]



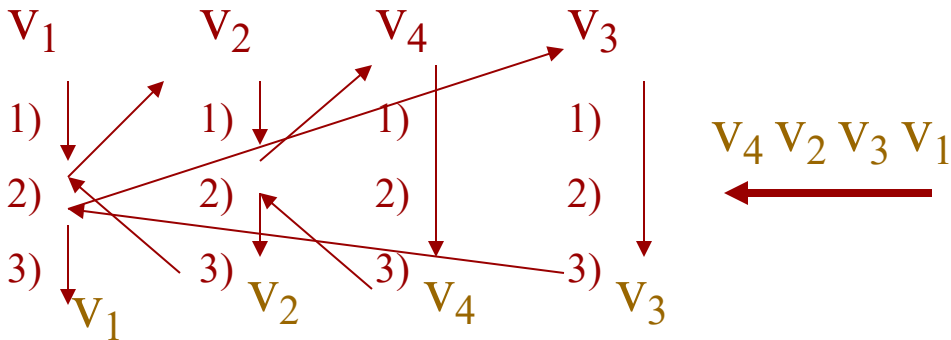
【方法二 算法思想】

(利用深度优先遍历)



- 1)访问顶点v，并记录v已被访问
- 2)依次从v的未访问的邻接点出发，深度优先拓扑排序图G。
- 3)（递归返回时）输出顶点v（此时v相当于无后继的顶点）

结论：用DFS遍历一个无环有向图，并在DFS算法退栈返回时打印相应的顶点，则输出的顶点序列是逆拓扑序列



【方法二适用范围】

有向无环图

(此算法不能检测出有向环，得到虚假拓扑序列)

3 关键路径

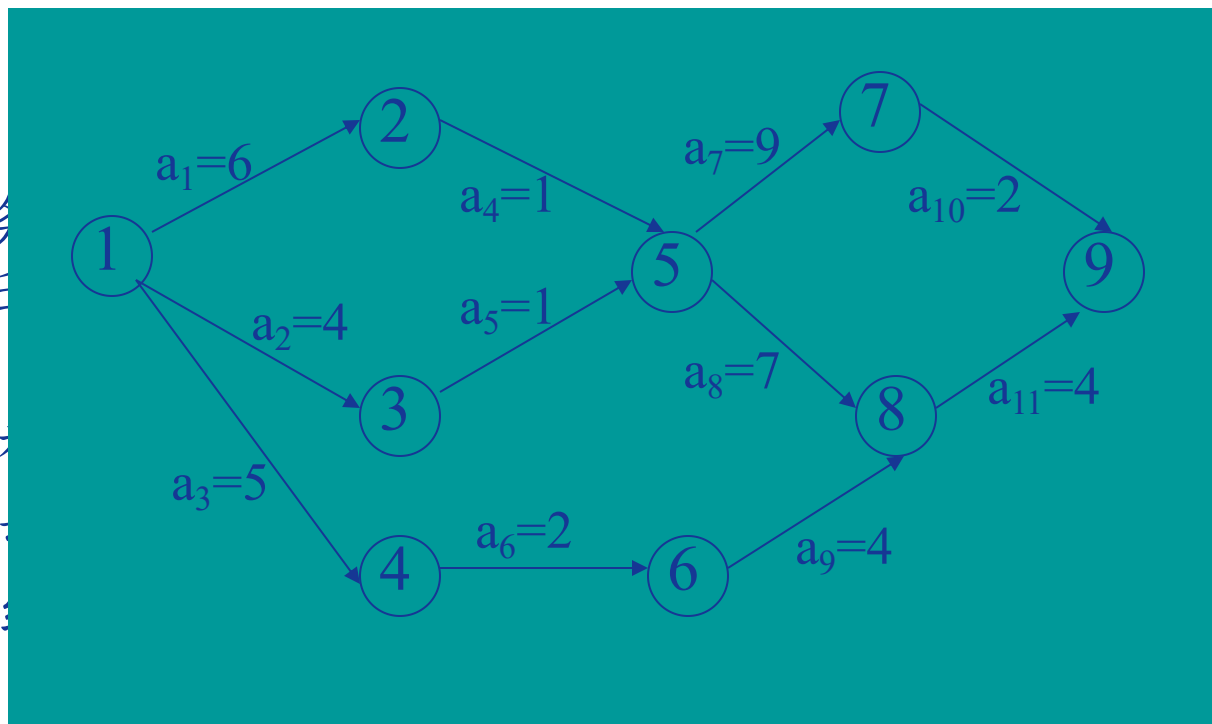
[问题目标]

当一个任务
工程，其中的任
先决条件

整个工

哪些活

AOE网(边活动网)



[AOE网的性质]

- 进入 V_i 的活动都结束， V_i 所代表的事件才能发生
- 顶点 V_i 所代表的事件发生，从 V_i 出发的活动才能开始

[相关概念和术语]

源点 入度为0的顶点，即工程的开始点

汇点 出度为0的顶点，即工程的完成点

关键路径 从源点到汇点的最长路径

关键路径长度=最短工期

关键活动 关键路径上的活动 $e(i)=l(i)$

关键活动的加快可以缩短工期

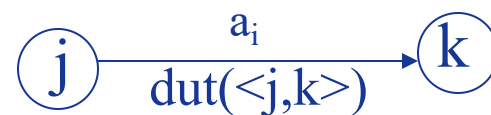
[确定关键路径时涉及的几个变量]

$e(i)$:活动 a_i 的最早开始时间

$l(i)$:活动 a_i 的最迟开始时间

$ve(j)$:事件 v_j 的最早发生时间

$vl(j)$:事件 v_j 的最迟发生时间



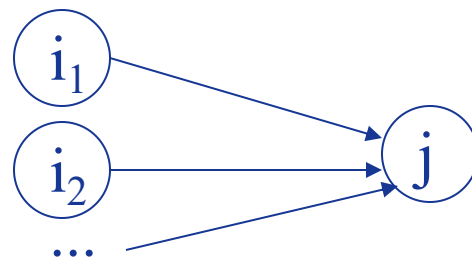
$$e(i)=ve(j)$$

$$l(i)=vl(k)-dut(<j,k>)$$

[关键路径的确定]

1) 求事件 v_j 的最早发生时间 $ve(j)$

$$\begin{cases} ve(1)=0 \\ ve(j)=\max\{ve(i)+dut(<i,j>)\} \\ (i \in j \text{的前驱}) \end{cases}$$

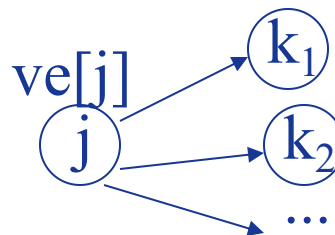


[算法思想]: 在求拓扑排序的过程中进行。

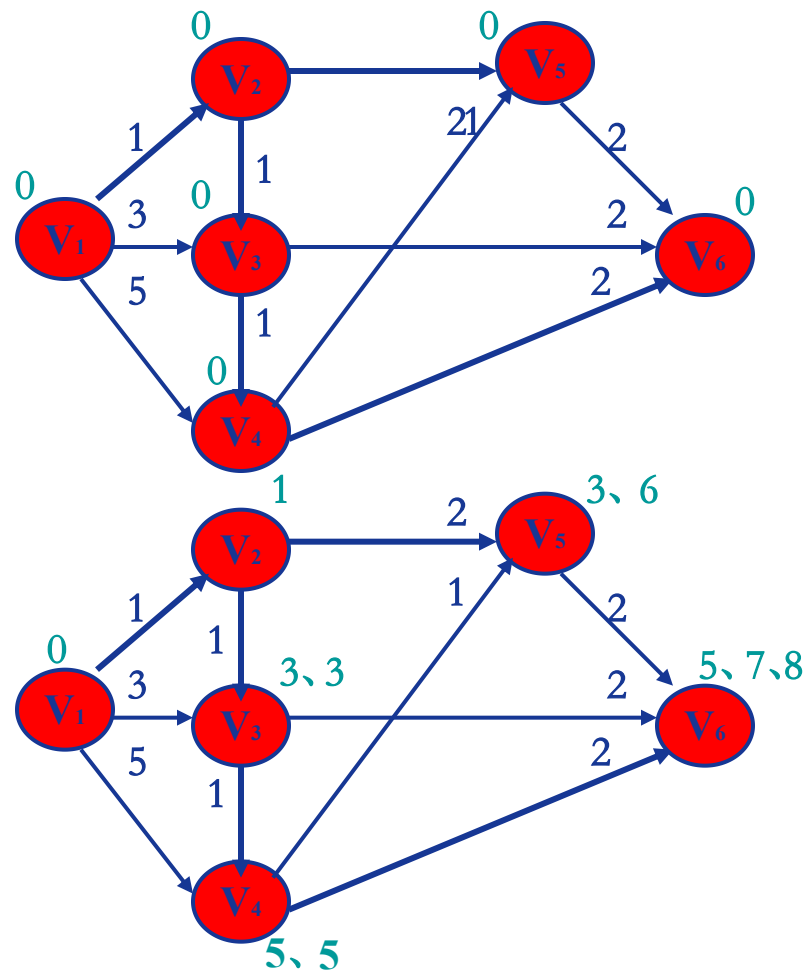
辅助数据结构: $ve(1..n)$ { n 为顶点个数}

1) 初值 $ve[j]=0 \quad 1 \leq j \leq n$

2) 每输出一个无前驱的顶点 j 后, 修正 j 的所有后继结点 k 的 $ve[k]$: $ve[k]=\max\{ve[k], ve[j]+dut(<j,k>)\}$

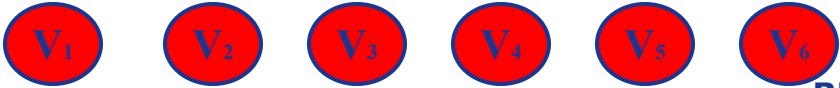


实例



- $Ve(1)=0$
- $Ve(2)=1$
- $Ve(3)=3$
- $Ve(4)=5$
- $Ve(5)=6$
- $Ve(6)=8$

正向拓扑排序:



```
bool Topologicalsort( Agraph *G, int &ve[]) //修改求拓扑序列算法，实现求ve值.
```

```
{
```

```
    findinDegree(G, indegree); //计算每个结点的入度，用indegree数组返回。
```

```
    Initstack(S); //栈S初始化操作
```

```
    for(i = 1; i <= G.n; i++)
```

```
    {
```

```
        if (indegree[ i ]==0) Push(S, i); //顶点i入度为0，入栈
```

```
        ve[i] = 0; //ve数组初始化为0
```

```
    }
```

```
    int count = 0;
```

```
    while (!StackEmpty(S)) //栈不空
```

```
    {
```

```
        Pop(S,j); //栈顶元素出栈，用j保存栈顶元素；
```

```
        ++count; //拓扑序列中元素个数加1；
```

```
        for (p=G->adjlist[j].firstarc; p; p=p->nextarc)//寻找顶点j的邻接顶点
```

```
        {
```

```
            k= p->adjvex;//顶点j的邻接顶点k;
```

```
            if ((- indegree [ k ]) ==0) Push(S, k); //顶点k入度-1之后若==0，则k入栈；
```

```
            if (ve[j]+ p->weight > ve[k]) ve[k] = ve[j] + p->weight; //更新顶点k的ve值
```

```
        }
```

```
    }
```

```
    if (count < G.n) return FALSE; //栈空，但输出顶点数不等于G.n，有环；
```

```
    else return TRUE; //无环，输出的就是拓扑序列
```

```
}
```

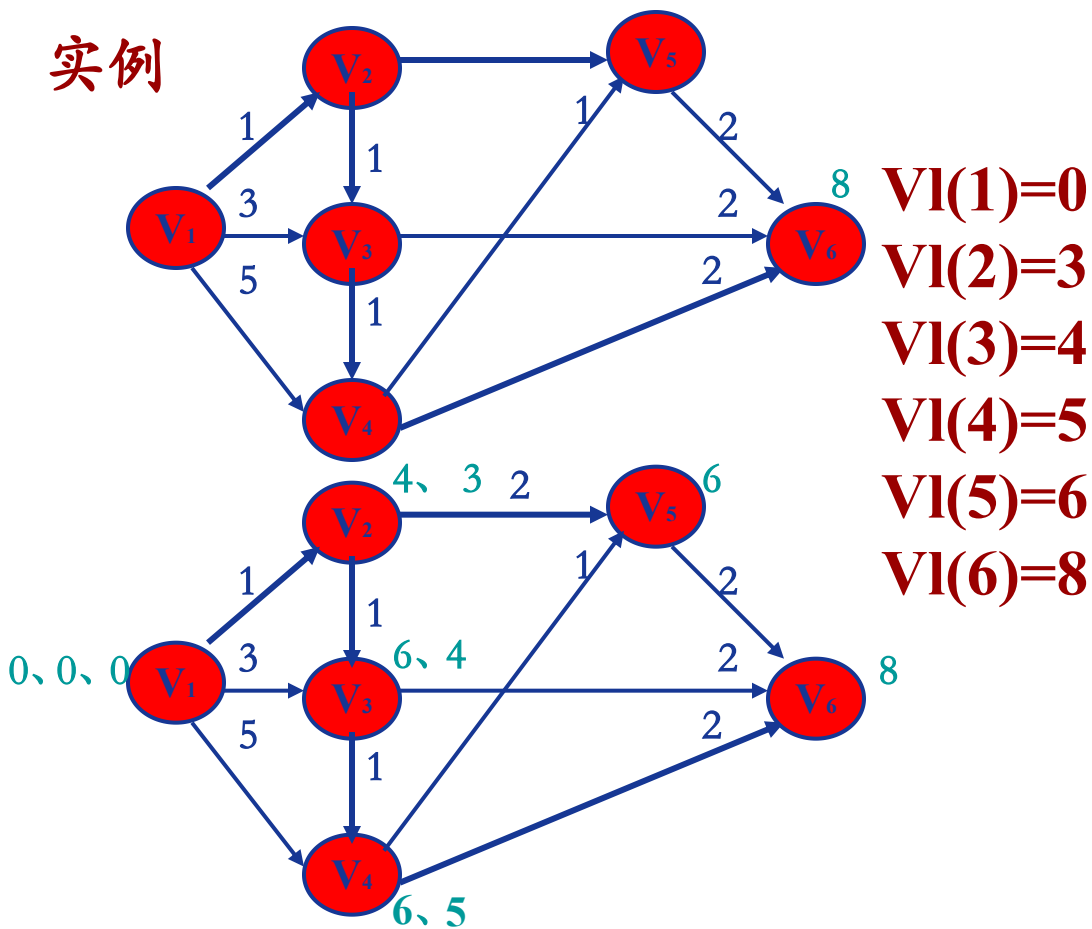
A diagram showing a node labeled j in a circle. Three arrows originate from node j and point to three nodes on the right: k_1 , k_2 , and an ellipsis ($\dots$), all of which are also in circles.

辅助数据结构: $vl(1..n)$ $\{n \text{ 为顶点个数}\}$

原理：拓扑序列 ○ ○ ○ j | ○ ○ ○

BUPT SCST

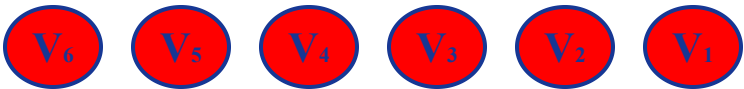
实例



利用逆向拓扑排序算法求事件结点的最迟发生时间的执行步骤：

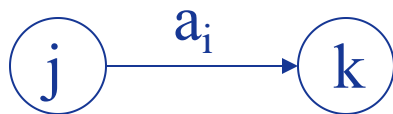
- 1、设每个结点的最迟发生时间为汇点的最早发生时间。
- 2、将栈中的结点V取出。
- 3、根据逆邻接表找到结点V的所有的起始结点，将结点V的最迟发生时间 - 活动的权值得到的差同起始结点的原最迟发生时间进行比较；如果该值小，则用该值取代原最迟发生时间。
- 4、反复执行 2、3；直至栈空为止。

逆向拓扑排序：



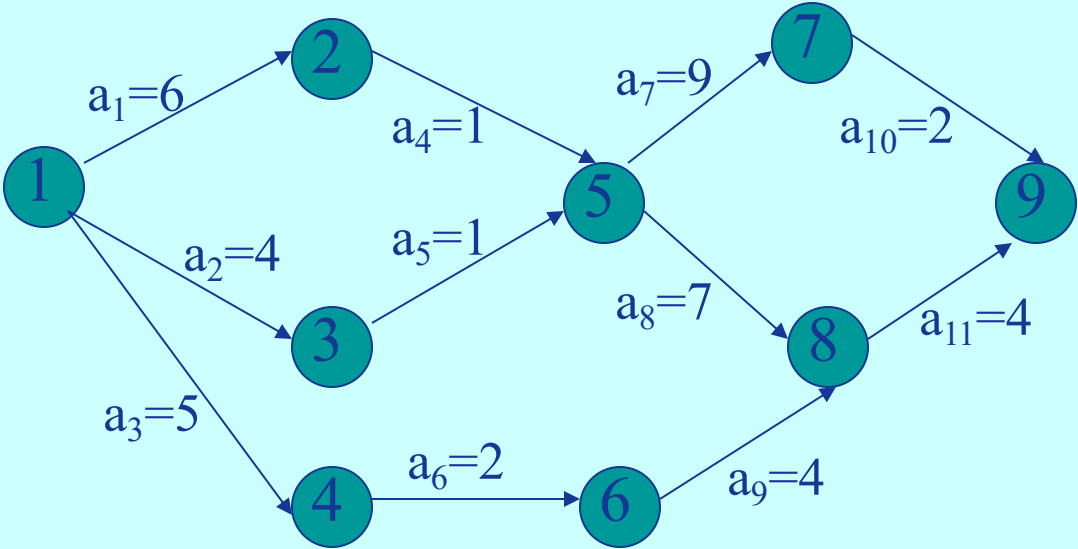
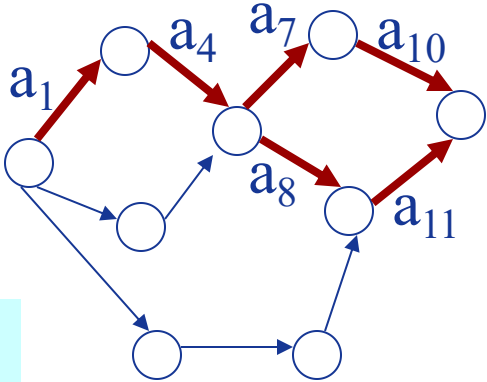
3) 求活动 a_i 的最早开始时间 $e(i)$ 和最迟开始时间 $l(i)$
确定关键活动 $e(i)=l(i)$

$$\begin{cases} e(i)=ve(j) \\ l(i)=vl(k)-dut(<j,k>) \end{cases}$$



j	1	2	3	4	5	6	7	8	9
ve(j)	0	6	4	5	7	7	16	14	18
vl(j)	0	6	6	8	7	10	16	14	18

i	1	2	3	4	5	6	7	8	9	10	11
e(i)	0	0	0	6	4	5	7	7	7	16	14
l(i)	0	2	3	6	6	8	7	7	10	16	14



两条关键路径
最短工期：18

[求关键路径算法步骤]

主数据结构：图—邻接矩阵或邻接表

辅助： $ve(1..n)$, $vl(1..n)$, $e(1..m)$, $l(1..m)$

{ n 为图的顶点数目； m 为图的弧数目}

s —存储拓扑排序的顶点序列

(1)输入所有弧的信息，建立AOE网的存储结构

(2)求拓扑排序的顶点序列

每求得一个顶点 j ，则使 j 入栈 s ，并调整 j 的所有后继的 ve 值

(3)按栈 s 的从栈顶到栈底的顺序求顶点的 vl 值

(4)求每个活动 i 的 $e(i)$ 和 $l(i)$ 值，确定关键路径

7.6 最短路径

BUPT

[定义]

给定带权有向图 $G=(V,E)$ 和权重函数 $w:E \rightarrow R$, 该权重函数将每条边映射到实数值的权重上。图中一条路径 $p=\langle v_0, v_1, \dots, v_k \rangle$ 的权重 $w(p)$ 是所有边的权重之和 $w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$ 。

顶点 u 到 v 的**最短路径** $\delta(u, v)$: $\delta(u, v) = \begin{cases} \min \{w(p): u \xrightarrow{p} v\}, & \text{存在 } u \text{ 到 } v \text{ 的路径} \\ \infty, & \text{不存在 } u \text{ 到 } v \text{ 的路径} \end{cases}$

单源点最短路径: 给定一个有向图 $G=(V,E)$, 求给定源点 $s \in V$ 到每个顶点 $v \in V$ 的最短路径。

最短路径的最优子结构: 两个顶点之间的最短路径包含着其他的最短路径。最短路径算法通常依赖于此性质。

引理1-最短路径的子路径也是最短路径: 给定带权有向图 $G=(V,E)$ 和权重函数 $w:E \rightarrow R$, 设 $p=\langle v_0, v_1, \dots, v_k \rangle$ 为从顶点 v_0 到 v_k 的最短路径, 且对 $\forall i, j, 0 \leq i \leq j \leq k$ $p_{ij}=\langle v_i, v_{i+1}, \dots, v_j \rangle$ 为路径 p 中从顶点 v_i 到 v_j 的子路径。

证明: 将路径 p 分解为 $v_0 \xrightarrow{p_{0i}} v_i \xrightarrow{p_{ij}} v_j \xrightarrow{p_{jk}} v_k$, 则 $w(p) = w(p_{0i}) + w(p_{ij}) + w(p_{jk})$ 。现假设存在一条

顶点 v_i 到 v_j 的路径 p'_{ij} , 使得 $w(p'_{ij}) < w(p_{ij})$, 则 $v_0 \xrightarrow{p_{0i}} v_i \xrightarrow{p'_{ij}} v_j \xrightarrow{p_{jk}} v_k$ 的路径权重为

$w(p_{0i}) + w(p'_{ij}) + w(p_{jk}) < w(p_{0i}) + w(p_{ij}) + w(p_{jk})$ 的路径, 是一条比 p 更短的路径, 与 p 是最短路径的假设矛盾。

BUPT SCST

[环路]

最短路径不包含权值为正或负的环路。

[单源点最短路径问题的变体]

■ 单源点最短路径问题

- ✓ Bellman-Ford算法-边的权值可为负，但不包含权值为负的环路。返回布尔值，如有负环路返回FALSE，表示无最短路径，否则求出最短路径
- ✓ DAG-Shortest-Path算法 – 用于有向无环图
- ✓ Dijkstra算法-边的权值不能为负
- 单目的地最短路径问题：求每个顶点 v 到给定目的顶点 t 的最短路径。将有向边指向翻转过来即单源点最短路径问题。
- 单顶点对最短路径问题：求给定顶点 u 到给定顶点 v 的最短路径。即单源点最短路径的简化问题。
- 所有结点对的最短路径问题：对每对顶点 u, v ，求 u 到 v 的最短路径。即对每个顶点求单源点最短路径。
- ✓ Floyd-warshall算法-负权重边可以存着，但不能存着负权重环路。
- ✓ Johnson算法-用于稀疏图，用Dijkstra算法或Bellman-Ford算法作为子程序

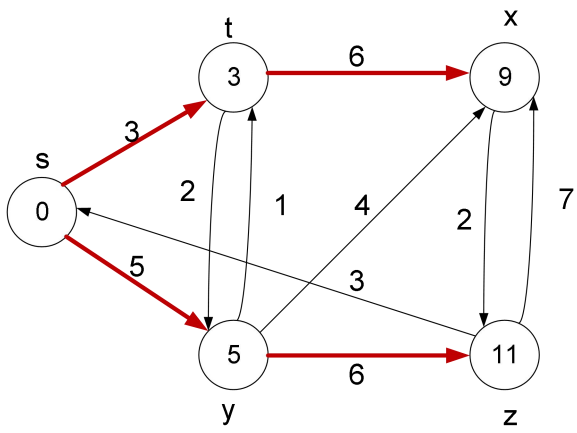
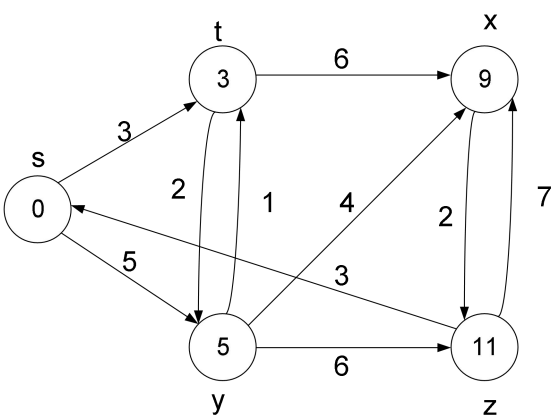
[最短路径的表示]

给定带权有向图 $G=(V,E)$ ，在最短路径算法中，每个顶点 $v \in V$ 维持一个前驱顶点 $p[v]$ ， $p[v]$ 可能是一个顶点也可能是空(-1)；将结点 v 开始的前驱顶点链反转过来就是 s 到 v 的最短路径。数组 $\text{int } p[\text{MAXV}]$ 为全局变量，初值 $p[v]=-1$ 。

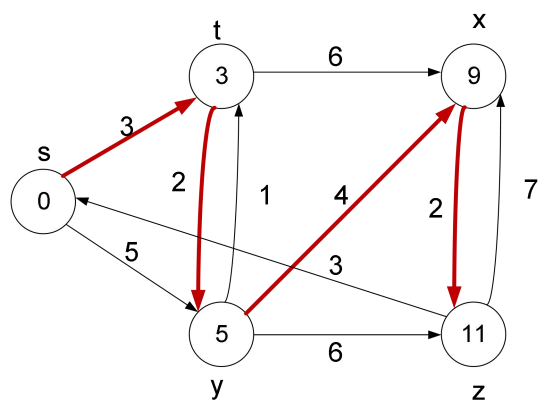
前驱子图 $G_p=(V_p,E_p)$ 是由 $p[v]$ 生成的，其中 V_p 是前驱顶点中不为空的顶点集合加上源顶点 s ，即 $V_p=\{v \in V: p[v] \neq -1\} \cup \{s\}$ ； E_p 由 V_p 中顶点通过 $p[v]$ 产生的边的集合，即： $E_p=\{ \langle p[v], v \rangle \in E: v \in V_p - \{s\} \}$ 。

在最短路径算法终止时， G_p 是一棵根结点为源顶点 s 的最短路径树，它包含：源顶点 s ，以及 s 到每个可达顶点 v 的一条最短路径。

最短路径可能不唯一，最短路径树也不唯一。



$s-t-y-x-z$ 是最短路径，则子路径 $t-y-x$ 也是最短路径



[最短路径的原理-松弛操作]

说明：在最短路径算法运行过程通过2个全局数组中每个结点维护2个属性：

- ① $\text{int } d[1, \dots, \text{MAXV}]$; $d[v]$ 记录通过当前从源顶点 s 出发到顶点 v 的最短路径估计，初值为无穷。
- ② $\text{int } p[1, \dots, \text{MAXV}]$; $p[v]=u$ ；这表示顶点 u 是顶点 v 在当前最短路径估计的前驱顶点；初值为 -1 表示无前驱顶点

松弛操作 **Relax** 过程：

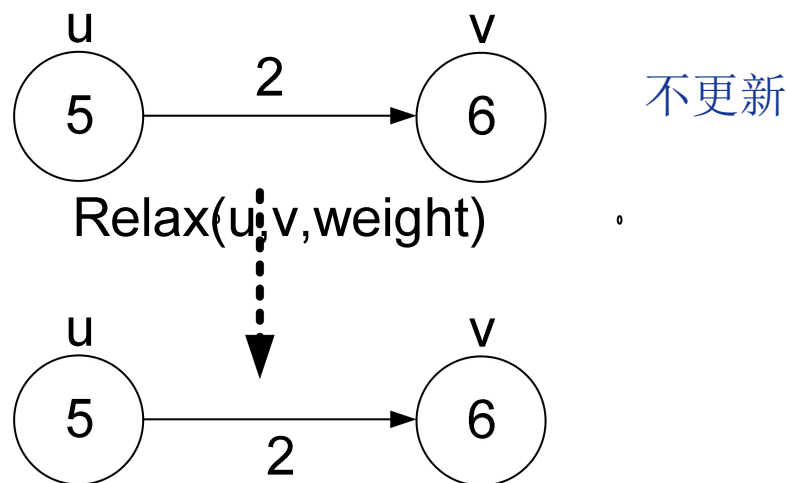
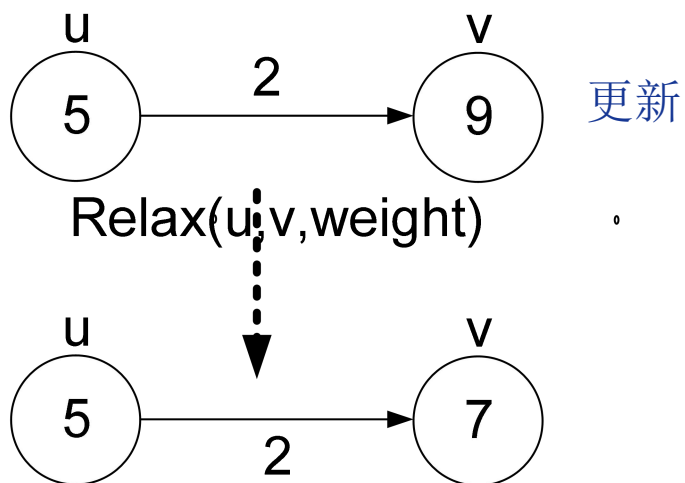
- ① 比较大小： s 到 u 之间的最短路径距离 $d[u] + \langle u, v \rangle$ 的权值 weight 之和与 s 到 v 之间的最短路径估计 $d[v]$ 。
- ② 若前者大，则更新 $d[v]$ 为 $d[u] + \text{weight}$, $p[v]$ 更新为 u 。

初始化属性：

`Initialize-Single-Source(Graph G, int s)` // 源顶点 s

```
{
    for (each  $v \in V$ )
    {
         $d[v] = \text{INF}$ ; // 无穷
         $p[v] = -1$ ; // 表示无前驱顶点
    }
}
```

```
void Relax(int u, int v, int weight)
{ // 有向边  $\langle u, v \rangle$  的权值  $\text{weight}$ 
  if(  $[d[v] > d[u] + \text{weight}]$  )
  {
     $d[v] = d[u] + \text{weight}$ ;
    //  $s \rightarrow v$  最短路径估计更新
     $p[v] = u$ ; //  $v$  前驱顶点更新为  $u$ 
  }
}
```



单源点最短路径算法的基本流程和特点:

1. 先调用 $\text{Initialize-Single-Source}(G, s)$ 进行初始化。
2. 重复调用 $\text{Relax}(u, v, \text{weight})$ 对边进行松弛，松弛操作是唯一导致最短路径估计 $d[v]$ 和前驱顶点 $p[v]$ 发生变化的操作。
3. 各单源点最短路径松弛算法不同之处在于对每条边松弛的次数和边的松弛顺序不一样。
4. Dijkstra 和用于有向无环图的 DAG-Shortest-Path 算法只对每条边松弛一次。
5. Bellman-Ford 算法对每条边松弛 $n-1$ 次， $n=|V|$ 。

[最短路径与松弛操作性质]

三角不等式性质：对任何边 $\langle u, v \rangle \in E$ 其权值为 $w(u, v)$ ，则有 $\delta(s, v) \leq \delta(s, u) + w(u, v)$ 。

上界性质：对源顶点 s 和任意顶点 $v \in V$ ，总有 $d[v] \geq \delta(s, v)$ ，一旦 $d[v]$ 到达 $\delta(s, v)$ ，则值不再变化。

非路径性质：若顶点 s 到顶点 v 之间不存在路径，则总有 $d[v] = \delta(s, v) = \infty$ 。

收敛性质：对任意顶点 $u, v \in V$ ，若 $s \rightsquigarrow u \rightarrow v$ 是 s 到 v 的一条最短路径，若边 (u, v) 松弛操作前 $d[u] = \delta(s, u)$ ，则进行松弛操作 $\text{Relax}(u, v, w(u, v))$ 之后， $d[v] = \delta(s, v)$ 。

路径松弛性质：若 $p = \langle v_0, v_1, \dots, v_k \rangle$ 是从源顶点 $s = v_0$ 到顶点 v_k 的一条最短路径，且对 p 中边松弛顺序为 $\langle v_0, v_1 \rangle, \langle v_1, v_2 \rangle, \dots, \langle v_{k-1}, v_k \rangle$ ；则 $d[v_k] = \delta(s, v_k)$ 。且该性质的成立与任何其他边的松弛操作无关；即使这些边的松弛操作穿插与 p 中边的松弛操作之间。

前驱子图性质：对所有顶点 $v \in V$ ，一旦 $d[v] = \delta(s, v)$ ，则前驱子图 G_p 是一颗根为源顶点 s 的最短路径树。

7.6.1 单源点最短路径算法

BUPT

用于解决—**[Bellman-Ford算法]** 时间复杂度 $O(n*e)$

般情况下的最短路径问题，边可以是负值，但不能有负环。

```
bool Bellman-Ford(Graph G, function w, int s)
```

```
{
```

```
    Initialize-Single-Source(G, s);
```

```
    for (i= 1; i<=G.n-1; i++) // $O(n*e)$ 
```

```
        for (each  $\langle u,v \rangle \in E$ ) //边按固定顺序松弛
```

```
            Relax(u,v,w(u,v));
```

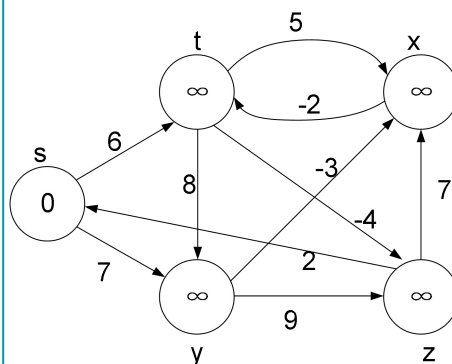
```
    for (each  $\langle u,v \rangle \in E$ )
```

```
        if( $d[v] > d[u] + w(u,v)$ ) //权重为负的环路
```

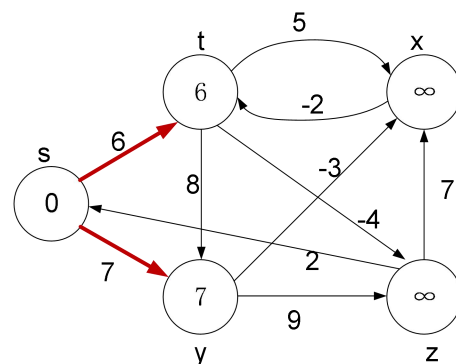
```
            return FALSE;
```

```
    return TRUE;
```

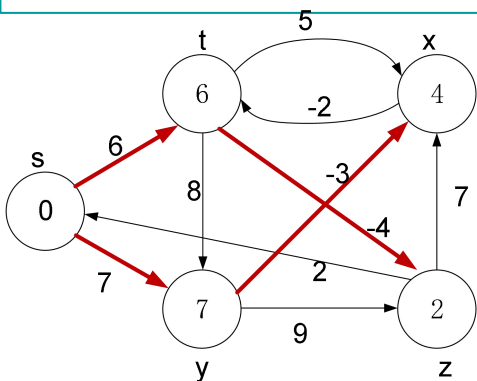
```
}
```



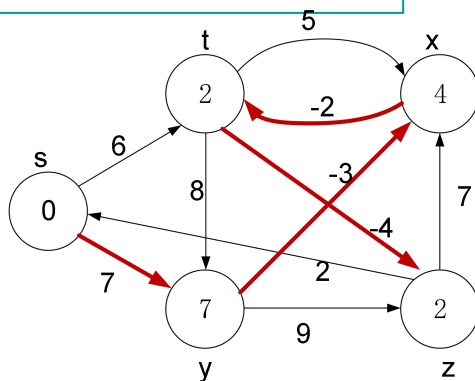
(a)



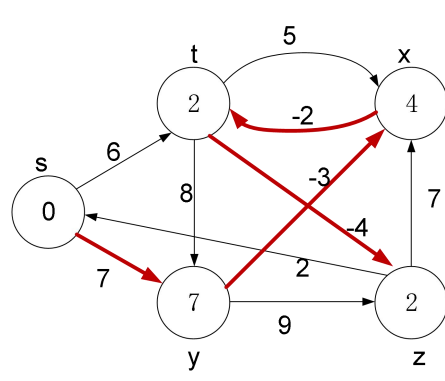
(b)



(c)



(d)



(e)

1. 源顶点s，每个顶点v中的数值为当前的 $d[v]$ 值，若 $\langle u,v \rangle$ 加红色，则 $p[v]=u$ 。
2. 共 $n-1=4$ 轮松弛，每轮松弛顺序均为 $\langle t,x \rangle, \langle t,y \rangle, \langle t,z \rangle, \langle y,x \rangle, \langle y,z \rangle, \langle z,x \rangle, \langle z,s \rangle, \langle s,t \rangle, \langle s,y \rangle$ 。
3. (a)第一轮松弛前；(b-e)每轮松弛后的场景。(e)最终场景。

[Bellman-Ford算法结论]

定理1: 给定带权有向图 $G=(V,E)$ 和权重函数 $w:E \rightarrow R$, 假定图 G 不包含从源顶点 s 可达的权重为负的环路, 则Bellman-Ford算法第2-4行的双重for循环执行 $n-1$ 次之后, 对于所有源顶点 s 可达的顶点 v , 有 $d[v]=\delta(s,v)$ 成立。

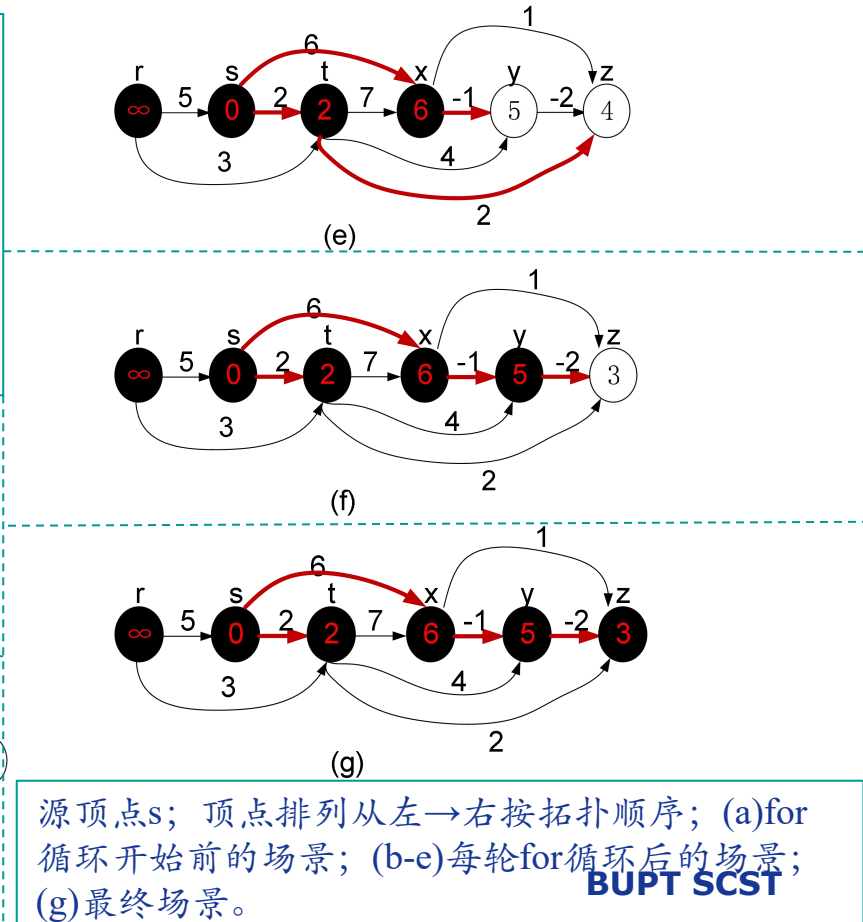
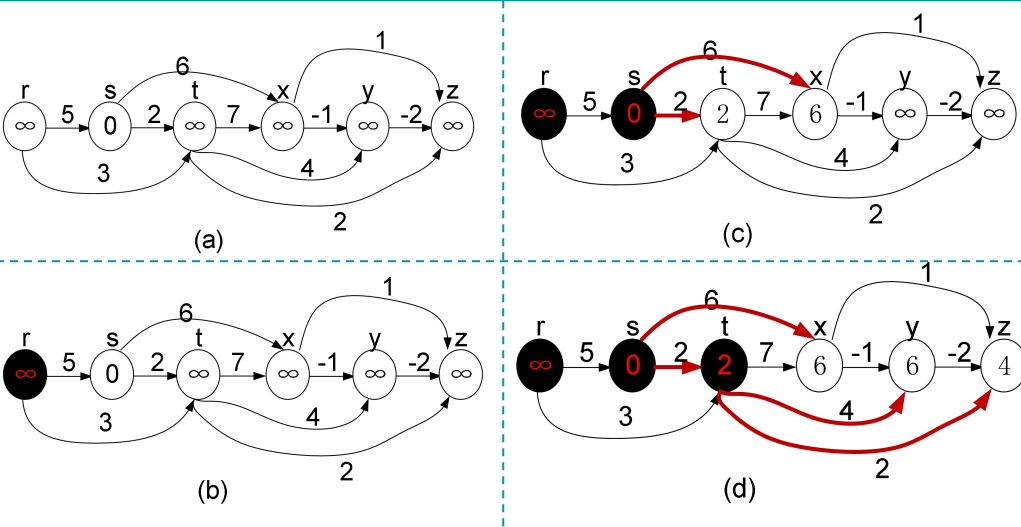
定理2: 给定带权有向图 $G=(V,E)$ 和权重函数 $w:E \rightarrow R$, 假定图 G 不包含从源顶点 s 可达的权重为负的环路, 则对于任一顶点 $v \in V$, 存在一条从源顶点 s 到 v 的路径, 当且仅当Bellman-Ford算法终止时 $d[v] < \infty$ 。

定理3: 在给定带权有向图 $G=(V,E)$ 和权重函数 $w:E \rightarrow R$ 上运行Bellman-Ford, 假定图 G 不包含从源顶点 s 可达的权重为负的环路, 则算法将返回TRUE; 且对于所有 $v \in V$, 前驱子图 G_p 是一颗根为源顶点 s 的最短路径树。如果图 G 包含一条从源顶点 s 可达的权重为负的环路, 则算法返回FALSE。

[**DA**G-Shortest-Path算法] 时间复杂度: $O(n+e)$

基本思想: 按照**拓扑顺序**对有向无环图G进行边的松弛操作。若 $\langle u,v \rangle$ 存在, 则顶点u在拓扑序列中必位于顶点v之前, 按拓扑序列顺序, 每次对一个顶点的**出边**进行松弛操作, 这样每条边只需处理一次即可。

```
DA
```



源顶点s; 顶点排列从左→右按拓扑顺序; (a)for 循环开始前的场景; (b-e)每轮for循环后的场景; (g)最终场景。

[DAG-Shortest-Path算法结论]

定理1： 给定带权有向无环图 $G=(V,E)$ 和权重函数 $w:E \rightarrow R$ ，假定源顶点为 s ，则在DAG-Shortest-Path算法终止时，对于所有顶点 $v \in V$ ，有：(1) $d[v]=\delta(s,v)$ 成立；(2) 其前驱子图 G_p 是一颗根为源顶点 s 的最短路径树。

定理2： 给定带权有向无环图 $G=(V,E)$ 和权重函数 $w:E \rightarrow R$ ，若边无负数，图 G 中边描述活动，边上权重表示活动所需时间，顶点表示事件；则可利用DAG-Shortest-Path算法**求关键路径**，方法为如下两种任一：

- (1) 将所有权重变为负数，然后运行DAG-Shortest-Path算法；
- (2) 运行DAG-Shortest-Path算法，但做如下改动：在Initialize-Single-Source(G, s)算法中第二行 $d[v] = \infty$ ；改为 $d[v] = -\infty$ ，即所有 $d[v]$ 初值由 $+\infty \rightarrow -\infty$ 。

[Dijkstra算法]

[预备知识-优先队列]

优先队列支持两个操作：

- ✓ 插入操作Insert(H)：等价于Enqueue(入队)操作；
- ✓ 删除并返回队列H中关键字最小者DeleteMin(H)操作，该操作替换了出队操作。

优先队列应用于求包含最小关键字的数据元素的场景；比如：多种排序算法、操作系统、贪心(婪)算法（反复求最小数据元素），和基于贪心思想的所有其他算法。



优先队列基本模型

[Dijkstra算法基本原理]

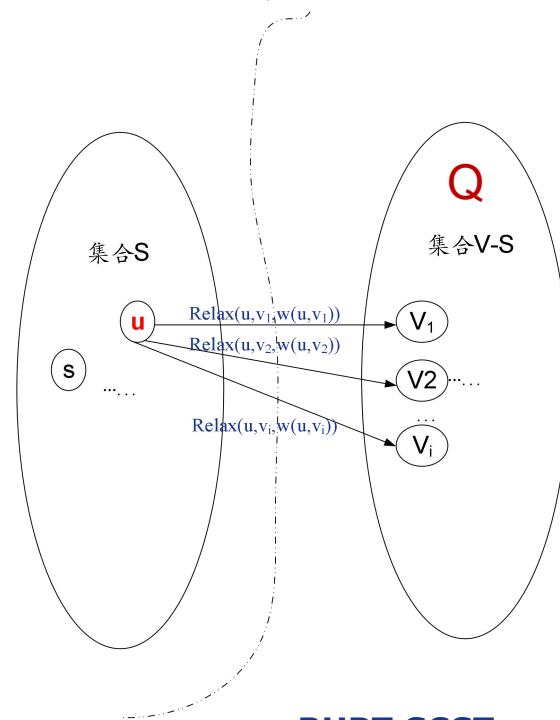
说明：Dijkstra算法要求边的权值为非负值。在算法运行过程中维护顶点集合 S ， S 的含义：从源顶点 s 到每个顶点 $v(v \in S)$ 之间的最短路径已确定。

基本思想：Dijkstra算法重复以下两步操作：

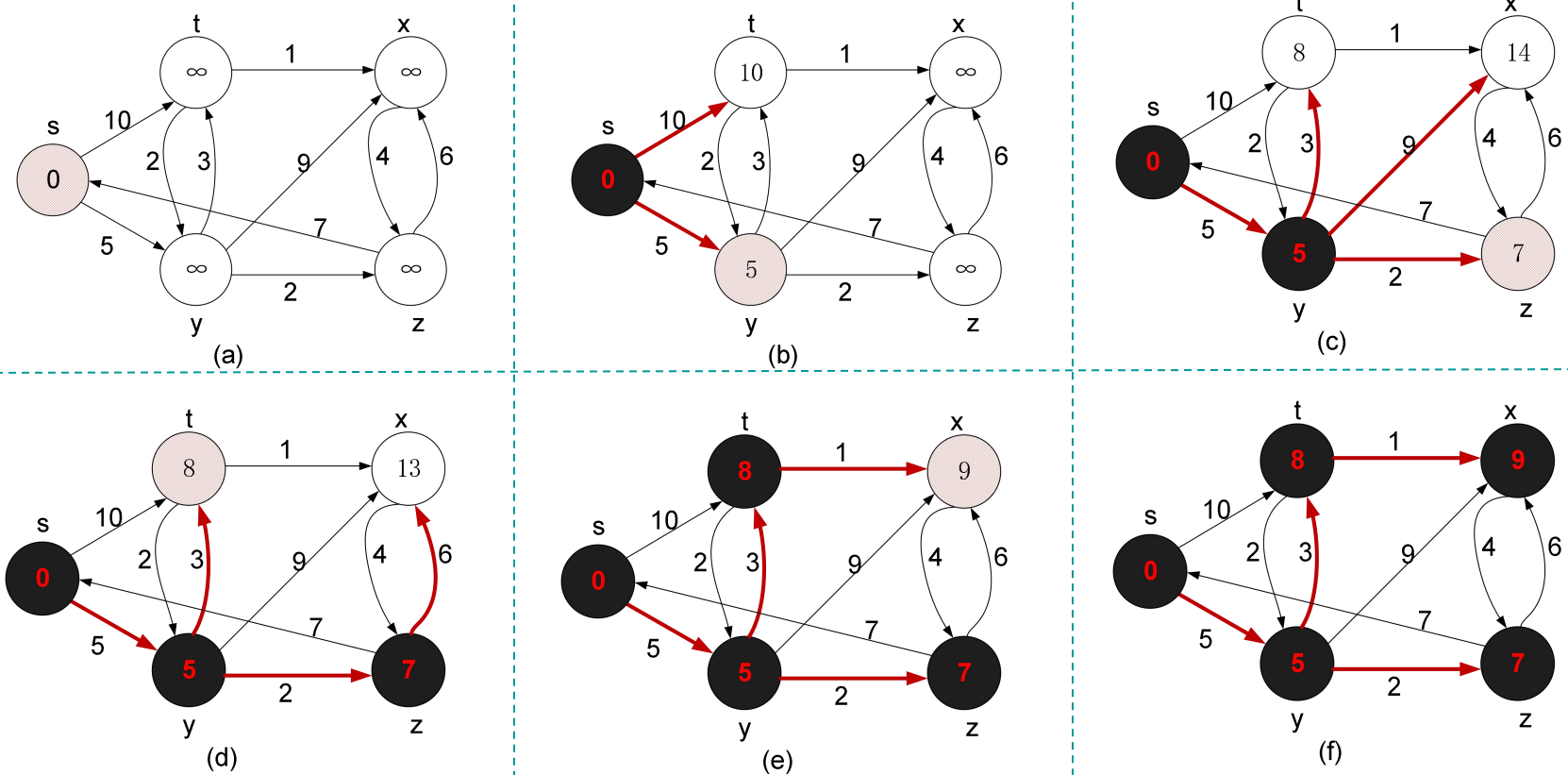
- (1) 从 $V-S$ 集合中寻找最短路径估计最小的顶点 u ，将顶点 u 加入到集合 S 中。
- (2) 对 u 的所有出边(跨接 S 集合和 $V-S$ 集合)进行松弛操作； $V-S$ 集合即最小优先队列 Q 。

```
Dijkstra(Graph G, function w, int s)
```

```
{
  Initialize-Single-Source(G, s);
  S =  $\emptyset$ ;
  Q = V; // Q为顶点的优先队列，关键字为每个顶点v的d[v]值。
  while (Q !=  $\emptyset$ ) // 队列不为空， $Q == V - S$ 
  {
    u = DeleteMin(Q) // u的d[u]值最小，出队
    S = S  $\cup$  {u}; // 出队后并入集合S
    for (each  $\langle u, v \rangle \in E$  &&  $v \in V - S$ ) // v是u的邻接顶点且 $v \in V - S$ 
      Relax(u, v, w(u, v));
  }
}
```



[Dijkstra算法示例]



- (1) 源顶点s，每个顶点中的数值为该顶点的最短路径估计；
- (2) 红色边表示前驱关系，其起始顶点是终端顶点的前驱值p[v]；
- (3) 黑色顶点属于集合S，白色顶点属于最小优先队列Q=V-S；
- (4) 加阴影的顶点为最小优先队列中d[]值最小的顶点；
- (5) (a)while循环执行前场景；(b-f)每轮循环后的场景；(f)中的d值和p值都是最终值。

[Dijkstra算法实现-邻接矩阵] 时间复杂度 $O(n^2)$

```

Void Dijkstra(Mgraph g, int s) //求顶点s到其他顶点的最短路径
{
    int S[1,..,MAXV]; //S数组存储集合S
    int Q[1,..,MAXV]; //Q数组存储优先队列Q
    int i,v,u =0;
    Initialize-Single-Source(g, s);  $O(n)$ 
    for (i= 1; i<=g.n i++) {  $O(n)$ 
        S[i] = -1; Q[i] =i; d[i] = g.edges[s][i]; // S= $\emptyset$ , Q=V
        if (d[i] < INF) p[i] = s;
        else p[i] =-1;
    } //end for
    while ( (u = DeleteMin(Q) ) != -1) {  $O(n^2)$ 
        //Q不为空(返回值不为-1), 即不是Q所有项都为-1,
        //并取d[]值最小的u,从队列Q中删除并返回
        S[u] = 1; //顶点u加入集合S
        for(v=1;v<=g.n; v++) { //取u的邻接顶点v
            if(S[v] == - 1) //若顶点v在V-S集合中;
                Relax(u,v, g.edges[u][v]); //边<u,v>做松弛操作
        } //end for
    } //end while
}

```

```

int DeleteMin(int Q[])
{
    int u =-1;
    for(int i=1;i<=MAXV;i++)
    {
        if(d[i] <d[i+1]&& Q[i]!=-1)
            u = i;
    }
    Q[u] = -1; //从Q中删除u
    return u;
}

```

Dijkstra算法时间复杂度取决于最小优先队列Q的实现。
若使用斐波那契堆实现最小优先队列，则算法时间复杂度可改善到 $O(n\lg n + e)$ ，对稀疏图而言，可明显提升运行速度。

[Dijkstra算法结论]

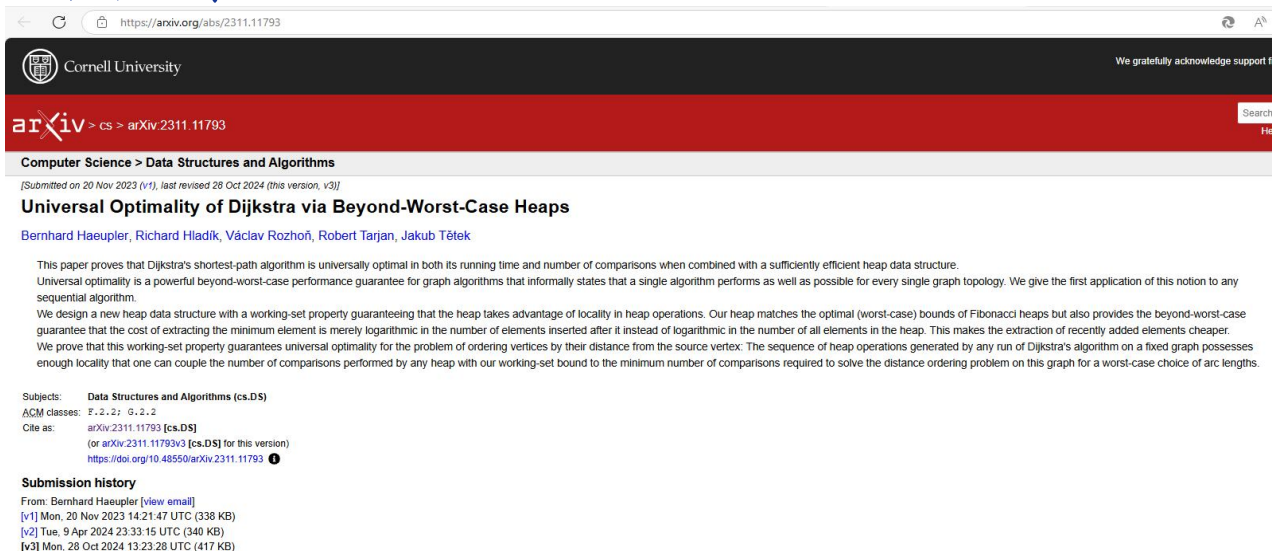
定理1: 给定带权有向无环图 $G=(V,E)$ ，若边无负权值；则对于所有顶点 $u(u \in V)$ ，Dijkstra算法终止时，有： $d[u]=\delta(s,u)$ 成立。

定理2: 若在带权有向无环图 $G=(V,E)$ 上运行Dijkstra算法，源顶点为 s ，且边无负权值；则算法终止时，其前驱子图 G_p 是一颗根为源顶点 s 的最短路径树。

Dijkstra被证明具有普遍最优性 (Universal Optimality)。

这说明即使在最坏情况下Dijkstra都能达到理论上的最优。

FOCS 2024(计算机顶会)
Best Paper



7.6.1 所有顶点对的最短路径算法

BUPT

[Johnson 算法/n次Dijkstra算法]

每次选一个顶点为源点，反复利用Dijkstra算法，要求：无负权重边。

时间复杂度： $O(n^3)$

[Floyd-Warshall/ Floyd算法]

采用动态规划，描述更简单，**负权重边可存在**，但不能有负权重环路。
时间复杂度： $O(n^3)$

[最短路径结构]

采用邻接矩阵存储图G，假设用矩阵 $\text{cost}[1\dots n][1\dots n]$ 存储边的权重， $\text{cost}[i][j]$ 存储边 $\langle i, j \rangle$ 的权重。若 $i=j$ ， $\text{cost}[i][j]=0$ ；若 $i \neq j$ ，且 $\langle i, j \rangle$ 不存在， $\text{cost}[i][j] = \infty$ 。

Floyd-Warshall考虑的最短路径上的中间结点。假设图G的顶点为 $V=\{1, 2, \dots, n\}$ ，考虑其中一个子集 $\{1, 2, \dots, k\} (k \leq n)$ ，假设对任意两个顶点 $i, j \in V$ ， $A^{(k)}$ 是路径权重矩阵， $A^{(k)}[i][j]$ 表示从顶点 i 出发，只经过序号小或等于 k 的中间顶点，到达顶点 j 的最短路径 p 的权重。

[最短路径规则]

①顶点i到顶点j的最短路径 $\delta(i,j)$ 可表示为： $\delta(i,j) = A^{(n)}[i][j]$ 。

② $A^{(0)}[i][j] = \text{cost}[i][j]$ 。

③若路径权重为 $A^{(k)}[i][j]$ 的最短路径p不经过顶点k，则 $A^{(k)}[i][j] = A^{(k-1)}[i][j]$ 。

④若路径权重为 $A^{(k)}[i][j]$ 的最短路径p经过顶点k，则该路径p可分解为两部分：(1)顶点i到顶点k的路径和(2)顶点k到顶点j的路径；由于路径(1)和(2)中间顶点都不包含顶点k，因此其路径权重分别为： $A^{(k-1)}[i][k]$ 和 $A^{(k-1)}[k][j]$ 。

基于上述规则，有：

$$A^{(k)}[i][j] = \min \{A^{(k-1)}[i][j], A^{(k-1)}[i][k] + A^{(k-1)}[k][j]\} \quad (k \geq 1)$$

$$A^{(0)}[i][j] = \text{cost}[i][j]$$

[最短路径上的前驱顶点]

$P^{(k)}$ 是前驱矩阵， $P^{(k)}[i][j]$ 表示从顶点*i*出发，只经过序号小或等于*k*的中间顶点，到达顶点*j*的最短路径*p*上，顶点*j*的前驱顶点。

$k=0$ ；顶点*i*到顶点*j*的路径不包含中间顶点，因此：

$$P^{(0)}[i][j] = \begin{cases} -1 & \text{若 } i = j \text{ 或 } w_{ij} = \infty \\ i & \text{若 } i \neq j \text{ 且 } w_{ij} < \infty \end{cases}$$

对于 $k \geq 1$ ，考虑路径*i* ~> *k* ~> *j* ($k \neq j$)，则选择的顶点*j*的前驱与路径*k* ~> *j* ($k \neq j$)的前驱顶点是一样的，这里：路径*k* ~> *j* ($k \neq j$)的中间顶点全部取自于集合{1, 2, ..., *k*-1}。否则，顶点*j*的前驱与路径*i* ~> *j*的前驱顶点是一样的，这里：路径*i* ~> *j*的中间顶点全部取自于集合{1, 2, ..., *k*-1}。也即：

$$P^{(k)}[i][j] = \begin{cases} P^{(k-1)}[i][j] & \text{若 } A^{(k-1)}[i][j] \leq A^{(k-1)}[i][k] + A^{(k-1)}[k][j] \\ P^{(k-1)}[\mathbf{k}][j] & \text{若 } A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j] \end{cases}$$

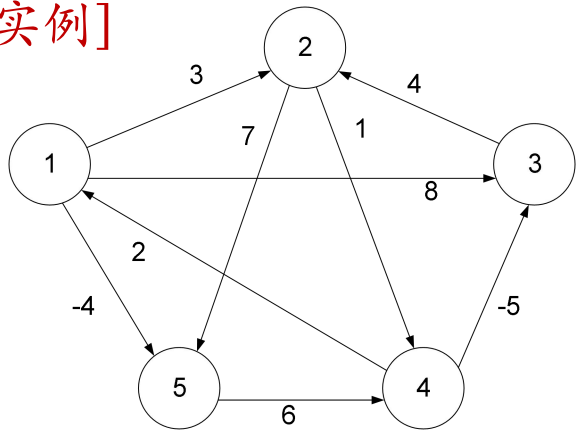
[算法描述]

```

int P[1..MAXV][1..MAXV]; //全局变量，二维数组P计算 $P^{(n)}[i][j]$ 
int A[1..MAXV][1..MAXV]; //全局变量，二维数组A计算 $A^{(n)}[i][j]$ 
#define NIL -1
void Floyd(int cost[][MAXV], int n)
{
    int i,j,k;
    for (i=1; i<=n; i++)
        for (j=1; j<=n; j++)
            {
                A[i][j] = cost[i][j];
                if(A[i][j]<INF && i!=j)  P[i][j] = i;
                else P[i][j] = NIL; //无前驱，置空
            }
    for (k=1; k<=n; k++)
        for (i=1; i<=n; i++)
            for (j=1; j<=n; j++)
                {
                    if (A[i][k]+ A[k][j])< A[i][j] )
                    {
                        P[i][j] = P[k][j];
                        A[i][j] = A[i][k]+ A[k][j];
                    }
                }
}

```

[算法实例]



$A^{(0)} = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$

$P^{(0)} = \begin{bmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & \text{NIL} & 4 & \text{NIL} & \text{NIL} \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{bmatrix}$

$A^{(1)} = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$

$P^{(1)} = \begin{bmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{bmatrix}$

$A^{(2)} = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$

$P^{(2)} = \begin{bmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{bmatrix}$

$A^{(3)} = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$

$P^{(3)} = \begin{bmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{bmatrix}$

$A^{(4)} = \begin{bmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$

$P^{(4)} = \begin{bmatrix} \text{NIL} & 1 & 4 & 2 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{bmatrix}$

$A^{(5)} = \begin{bmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$

$P^{(5)} = \begin{bmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{bmatrix}$

作业

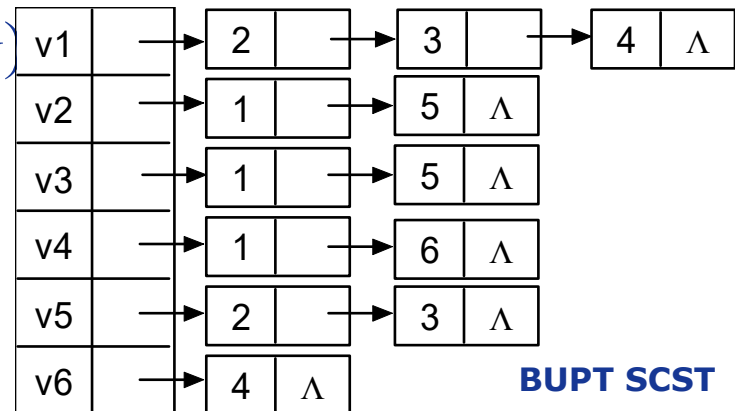
BUPT

1. 试证明：对于一个无向图 $G = (V, E)$ ，若 G 中各顶点的度均大于或等于 2，则 G 中必有回路。

2. 已知某图的邻接表如下图所示

- (1) 写出此邻接表对应的邻接矩阵；
- (2) 写出由 v_1 开始的深度优先遍历的序列；
- (3) 写出由 v_1 开始的深度优先的生成树；
- (4) 写出由 v_1 开始的广度优先遍历的序列；
- (5) 写出由 v_1 开始的广度优先的生成树；
- (6) 写出将无向图的邻接矩阵转换成邻接表的算法。

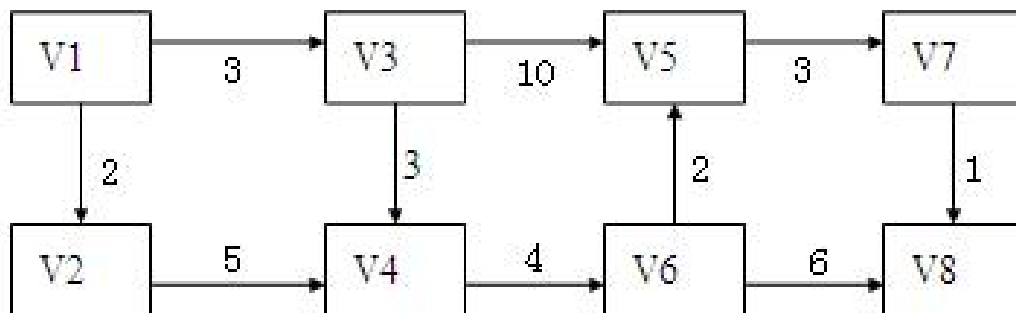
`void convert(Mgraph g, Agraph &*G)`



BUPT SCST

3. 已知一图如下图所示：

- (1) . 写出全部拓扑排序；
- (2) . 以v1为源点,以v8为汇点，给出所有事件允许发生的最早时间和最晚时间，并给出关键路径；
- (3) . 利用Dijkstra算法求V1结点到各点的最短距离。



4.有一图的邻接矩阵如下，试给出用Floyd算法求各点间最短距离的矩阵序列 A^1 , A^2 , A^3 , A^4 。

$$A = \begin{bmatrix} 0 & 2 & \infty & \infty \\ \infty & 0 & 1 & 6 \\ 5 & \infty & 0 & 4 \\ 3 & \infty & \infty & 0 \end{bmatrix}$$

5. 已知某图的邻接矩阵为A，即若从i到j有边，则 $A[i, j]=1$ ，否则 $A[i, j]=0$ 。试编写一算法求矩阵A的传递闭包C：使得若从i到j有一条或多条路径，则 $C[i, j]=1$ ，否则 $C[i, j]=0$ 。

```
typedef int adjmatrix [ maxvtxnum ] [ maxvtxnum ];
```

```
void change ( adjmatrix A, adjmatrix C, int n )
```

实验

BUPT

从命令行接受整数 V 和 E ,随机生成含有 V 个顶点和 E 条边的简单有向图,即不包含自环和平行边,存储结构无限定。对此有向图,运行实验求从一个随机选定的顶点可以到达的顶点数量的平均值。